

**BDBT – Spring Security do
zabezpieczenia aplikacji
webowej + Bootstrap (CSS)**

Tomasz Mrozek
Instytut Telekomunikacji
Wydział Elektroniki i Technik Informacyjnych



Wstęp

- Budowa prostej aplikacji Spring MVC zabezpieczonej stroną logowania. Mechanizm zabezpieczający aplikację został oparty o bibliotekę Spring Security.

UWAGA! W ramach obsługi perspektyw w drugiej części projektu, każda aplikacja powinna być wyposażona w bezpieczną stronę logowania.

Opracowano z wykorzystaniem:

<https://spring.io/guides/gs/securing-web/>

Zakres prezentacji

- Tworzenie projektu w Spring Initializr
- Tworzenie prostej aplikacji webowej ze stroną logowania
- Strona logowania
- Interfejs graficzny Bootstrap*
- Obsługa błędów w aplikacji

**Bootstrap – biblioteka CSS. Zawiera zestaw przydatnych narzędzi ułatwiających tworzenie interfejsu graficznego stron oraz aplikacji internetowych.*

1

TWORZENIE PROJEKTU W SPRING INITIALIZR

Tworzenie projektu w Spring Initializr

- Do rozpoczęcia projektu zostanie wykorzystany Spring Initializr dostępny na: <https://start.spring.io>.

Tworzenie projektu w Spring Initializr

- Należy określić następujące ustawienia projektu (w oknie Spring Initializr – następna strona):

Project:	Maven Project
Language:	Java
Spring Boot:	2.6.1
Dependencies:	Spring Web, Thymeleaf
Project Metadata:	
Group:	bdbt_bada_project
Artifact:	SpringApplication
Name:	SpringApplication
Description:	Sample project in Spring Boot for BDBT and BADA.
Package name:	bdbt_bada_project.SpringApplication
Packaging:	Jar
Java:	11

Tworzenie projektu w Spring Initializr

The screenshot shows the Spring Initializr web application interface. Handwritten orange annotations highlight key areas:

- 1**: A large box around the left-hand configuration section, including:
 - Project**: ☒ Maven Project, ☐ Gradle Project
 - Language**: ☒ Java, ☐ Kotlin, ☐ Groovy
 - Spring Boot**: ☐ 2.6.2 (SNAPSHOT), ☒ 2.6.1, ☐ 2.5.8 (SNAPSHOT), ☐ 2.5.7
 - Project Metadata**:
 - Group: bdbt_bada_project
 - Artifact: SpringApplication
 - Name: SpringApplication
 - Description: Sample project in Spring Boot for BDBT and BADA.
 - Package name: bdbt_bada_project.SpringApplication
 - Packaging**: ☒ Jar, ☐ War
 - Java**: ☐ 17, ☒ 11, ☐ 8
- 2**: A box around the **ADD DEPENDENCIES... CTRL + B** button in the Dependencies section.
- 3**: A box around the **GENERATE CTRL + G** button at the bottom.

Other visible elements include the Spring Initializr logo, a browser address bar with 'start.spring.io', and a footer with 'SpringApplication.zip' and a 'Pokaż wszystkie' link.

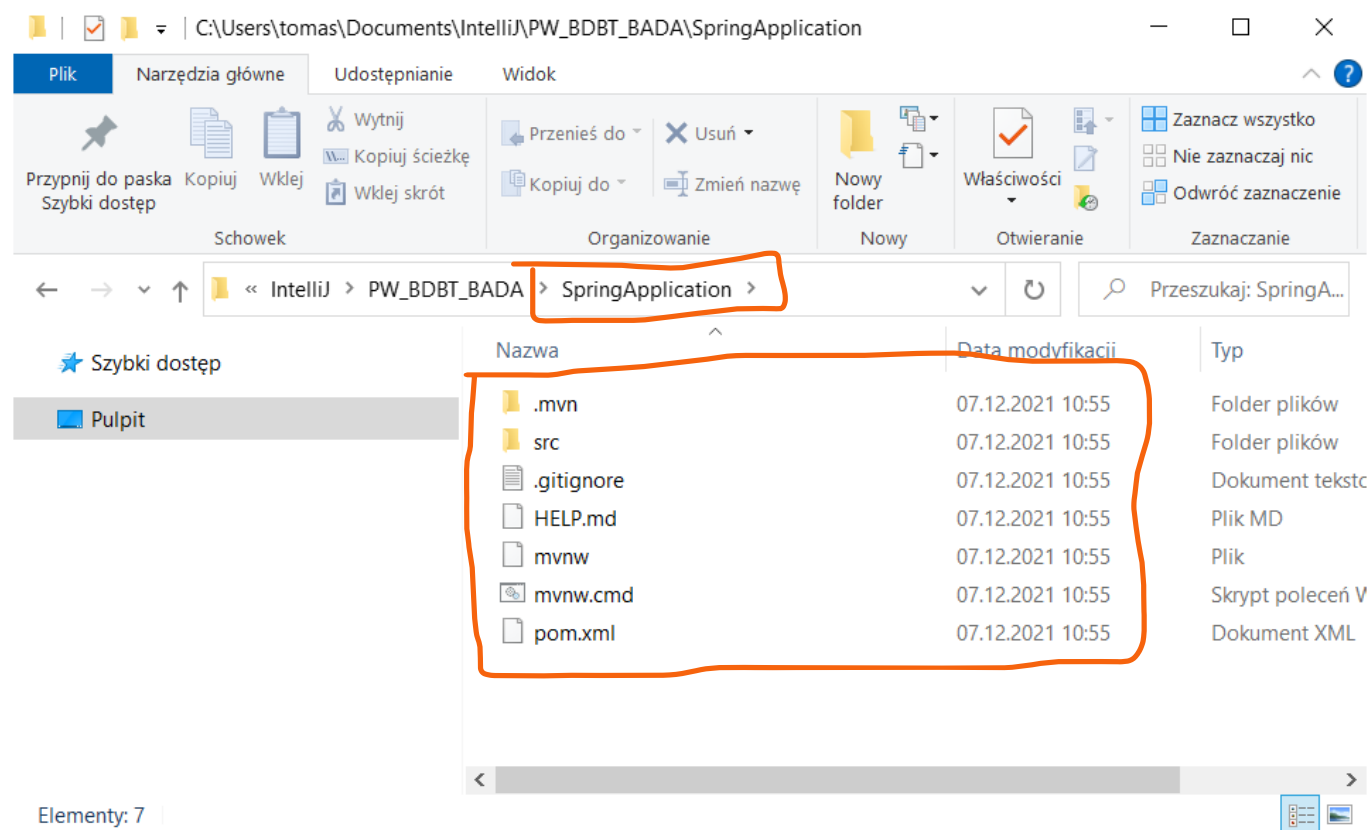
Tworzenie projektu w Spring Initializr

- Po wybraniu odpowiedni ustawień należy wybrać opcję Generate (czarny button pod wybranymi ustawieniami). Zostanie pobrany plik SpringApplication.zip. Należy go wypakować opcją „Wypakuj tutaj”.

UWAGA! Aby dodać zależności należy wybrać przycisk „Add dependencies”. Zależności można dodać na dowolnym etapie w pliku pom.xml.

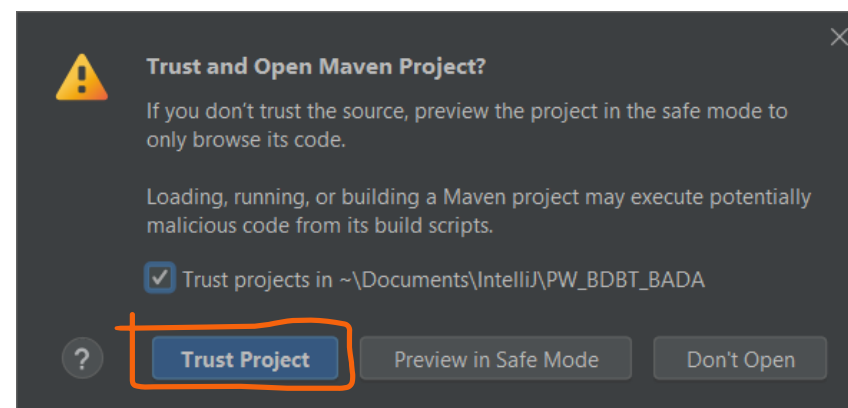
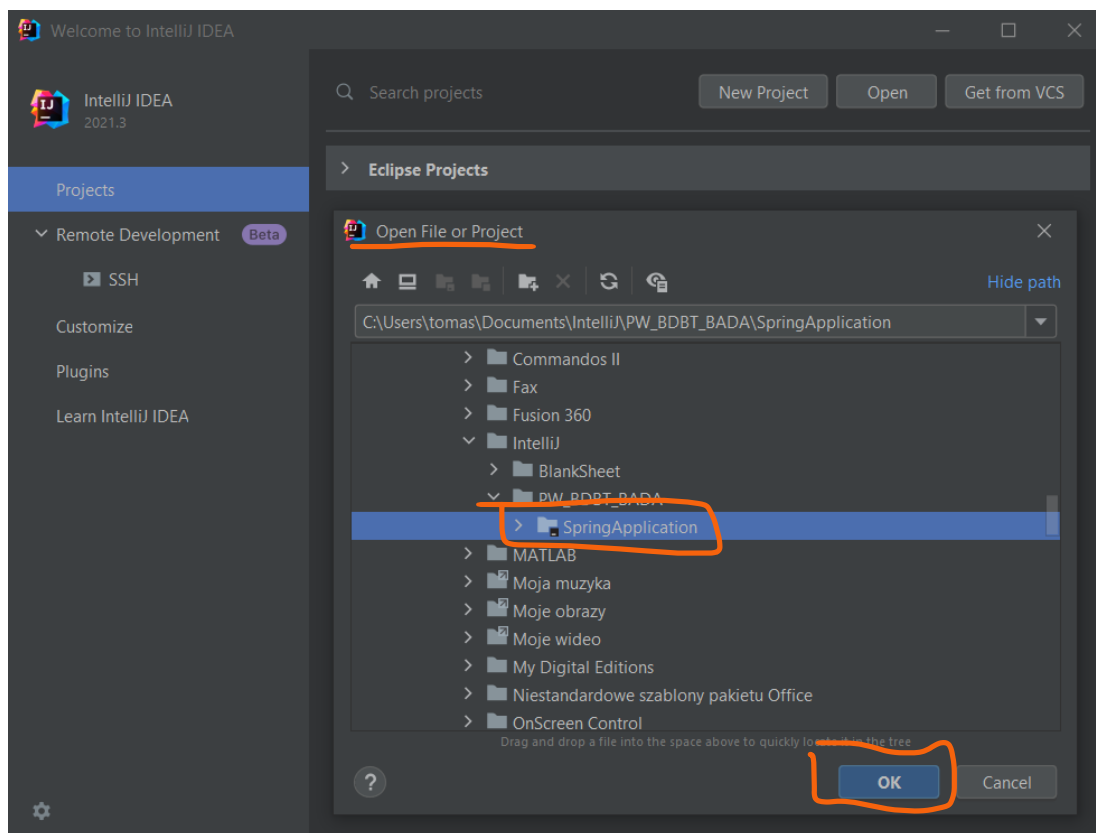
Tworzenie projektu w Spring Initializr

- Rozpakowany plik powinien mieć następującą zawartość:



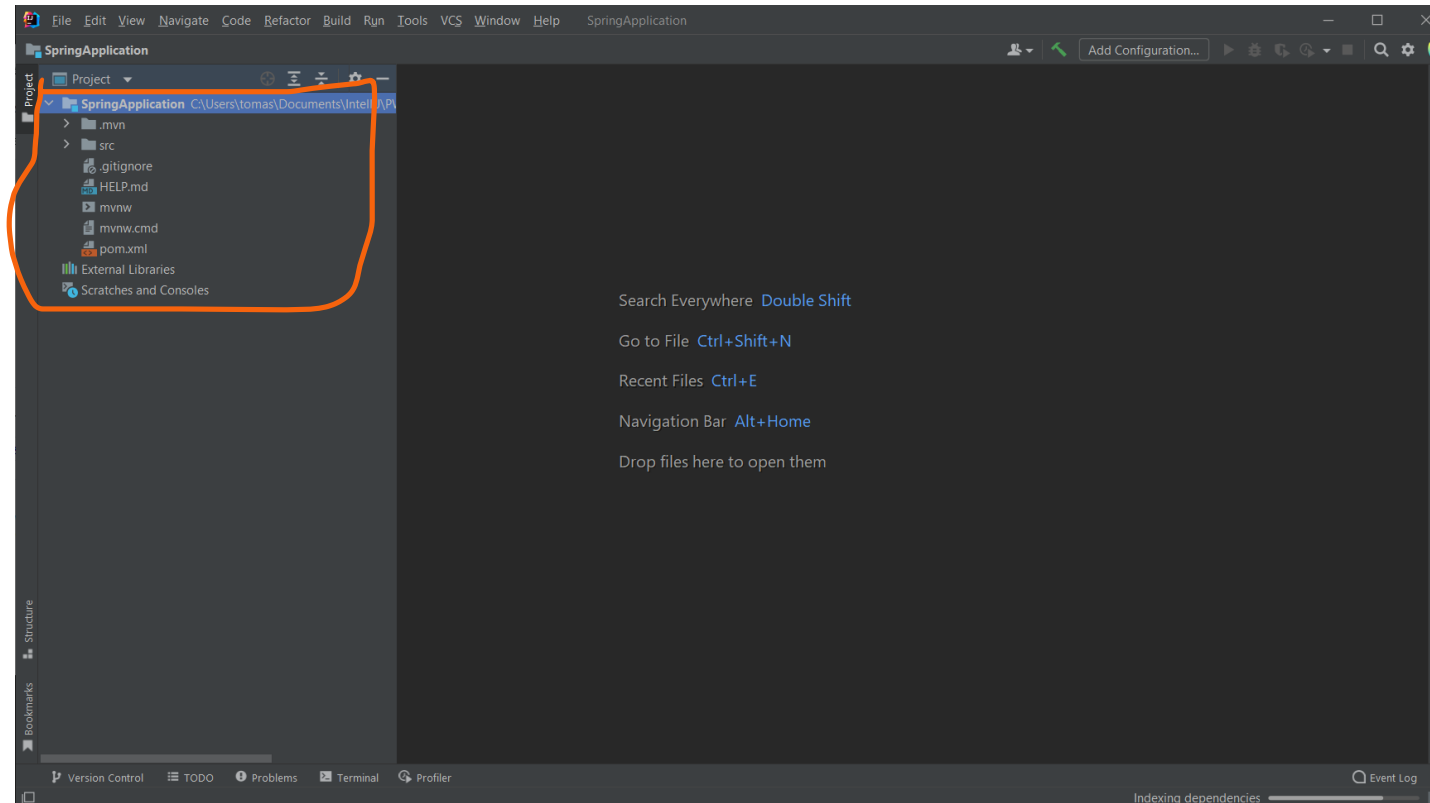
Tworzenie projektu w Spring Initializr

- Rozpakowany folder należy następnie otworzyć w programie IntelliJ:



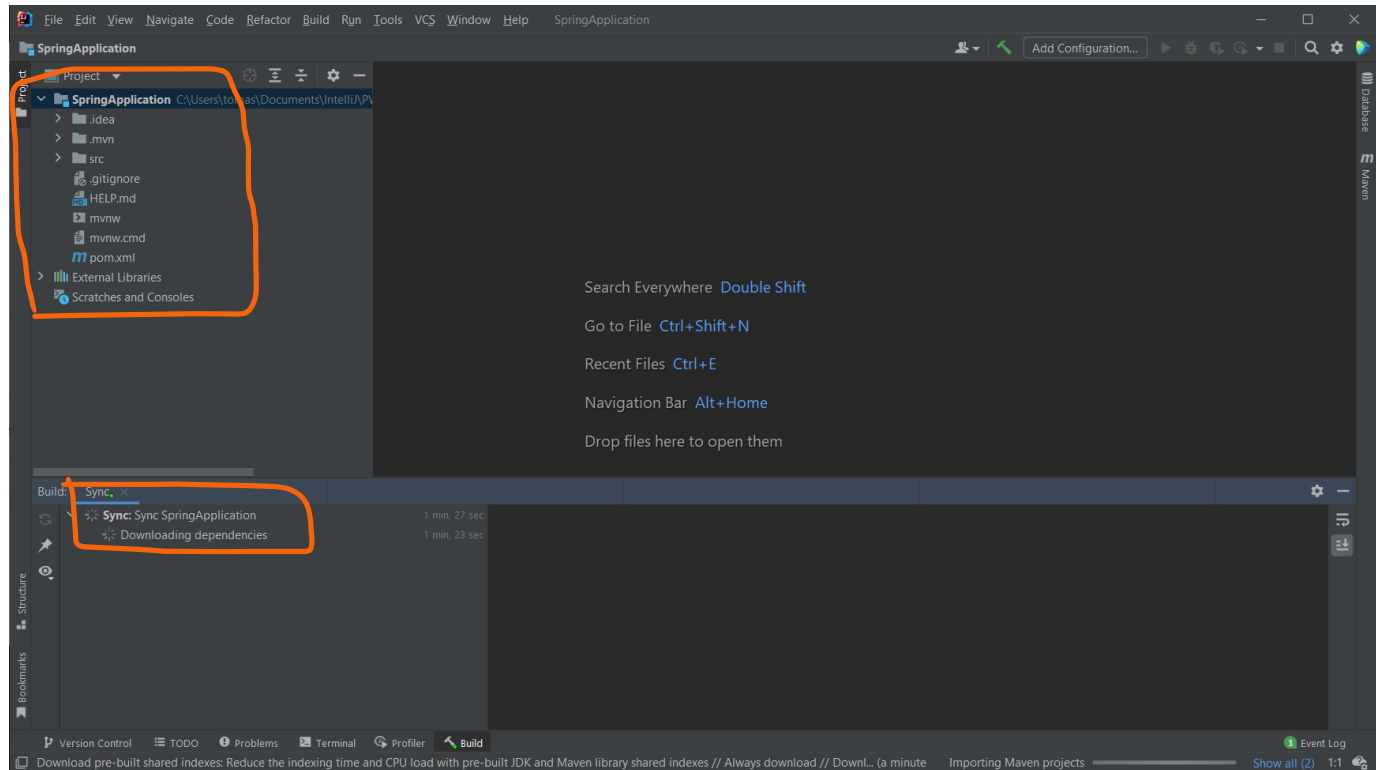
Tworzenie projektu w Spring Initializr

- Po otwarciu projekt powinien mieć poniższą strukturę:



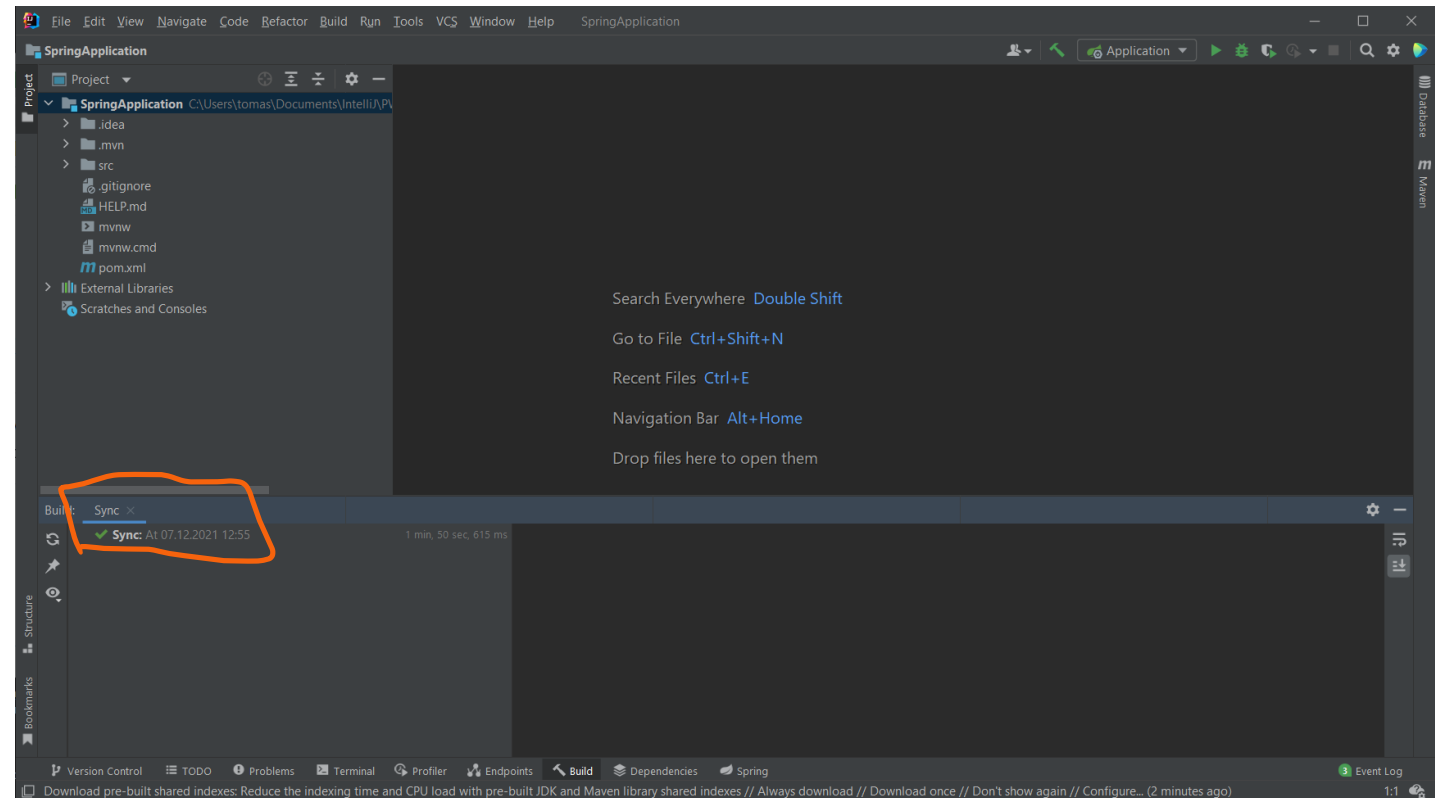
Tworzenie projektu w Spring Initializr

- Dodatkowo krótko po otwarciu nastąpi automatyczna synchronizacja plików. IntelliJ automatycznie pobierze wszystkie zależności dostępne w pliku pom.xml.



Tworzenie projektu w Spring Initializr

- Pomyślnie zakończona synchronizacja zostanie oznaczona zielonym znacznikiem.



2

TWORZENIE PROSTEJ APLIKACJI WEBOWEJ ZE STRONĄ LOGOWANIA

Tworzenie prostej aplikacji webowej ze stroną logowania

Pierwszym etapem jest stworzenie prostej aplikacji webowej, która na tym etapie nie będzie chroniona żadnymi hasłami.

Aplikacja webowa będzie zbudowana z dwóch prostych widoków:

- ze strony powitalnej (**index.html**), na której będzie się znajdować opcja logowania,
- ze strony głównej (**main.html**) dostępnej po zalogowaniu.

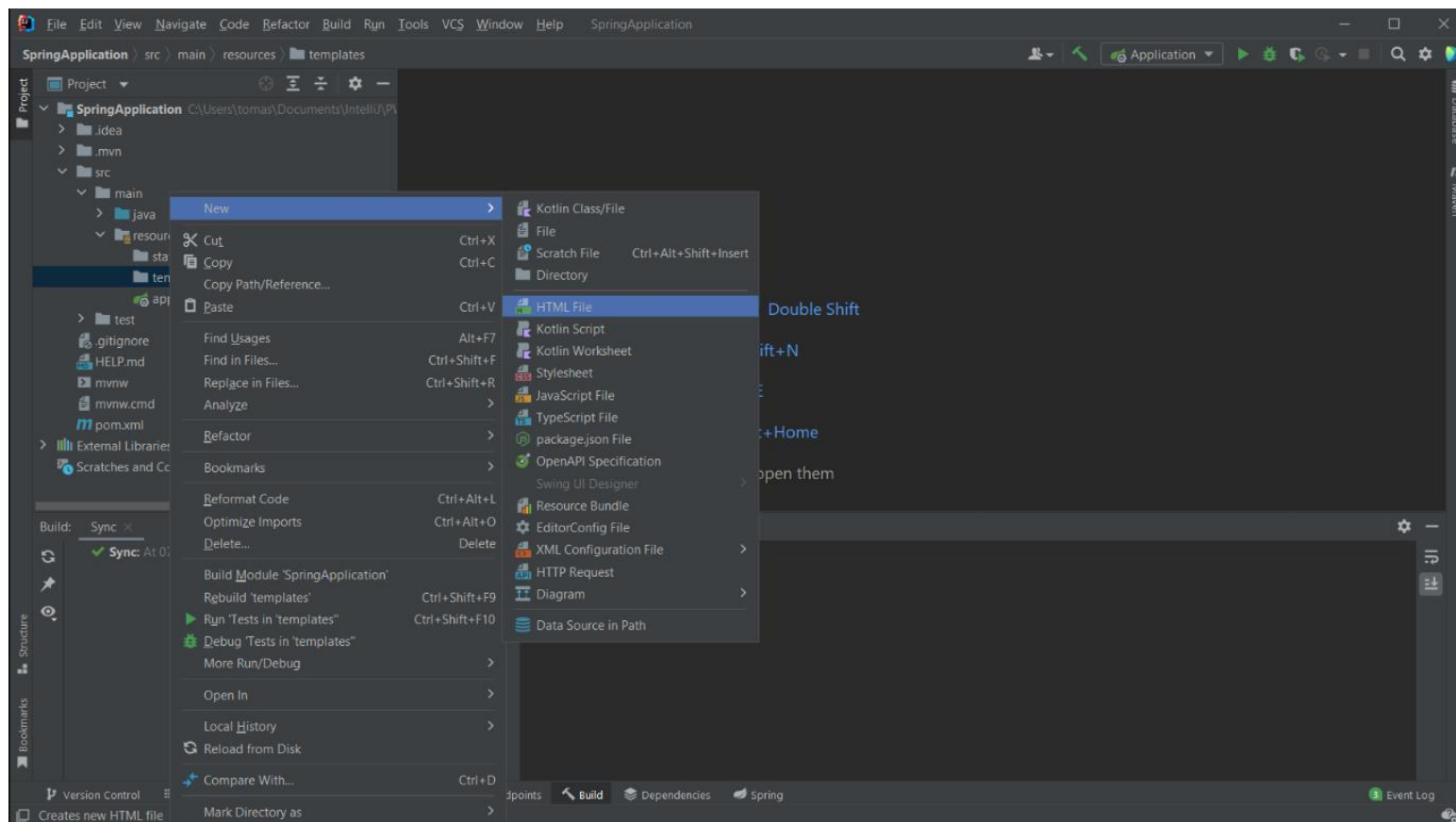
W dalszym etapie strony te zostaną rozbudowane.

Pliki .html odpowiedzialne za wyświetlanie widoków stron będą znajdować się w lokalizacji:
src/main/resources/templates/.

Ścieżka ta jest dostępna wraz z utworzeniem projektu i nie trzeba jej tworzyć od nowa.

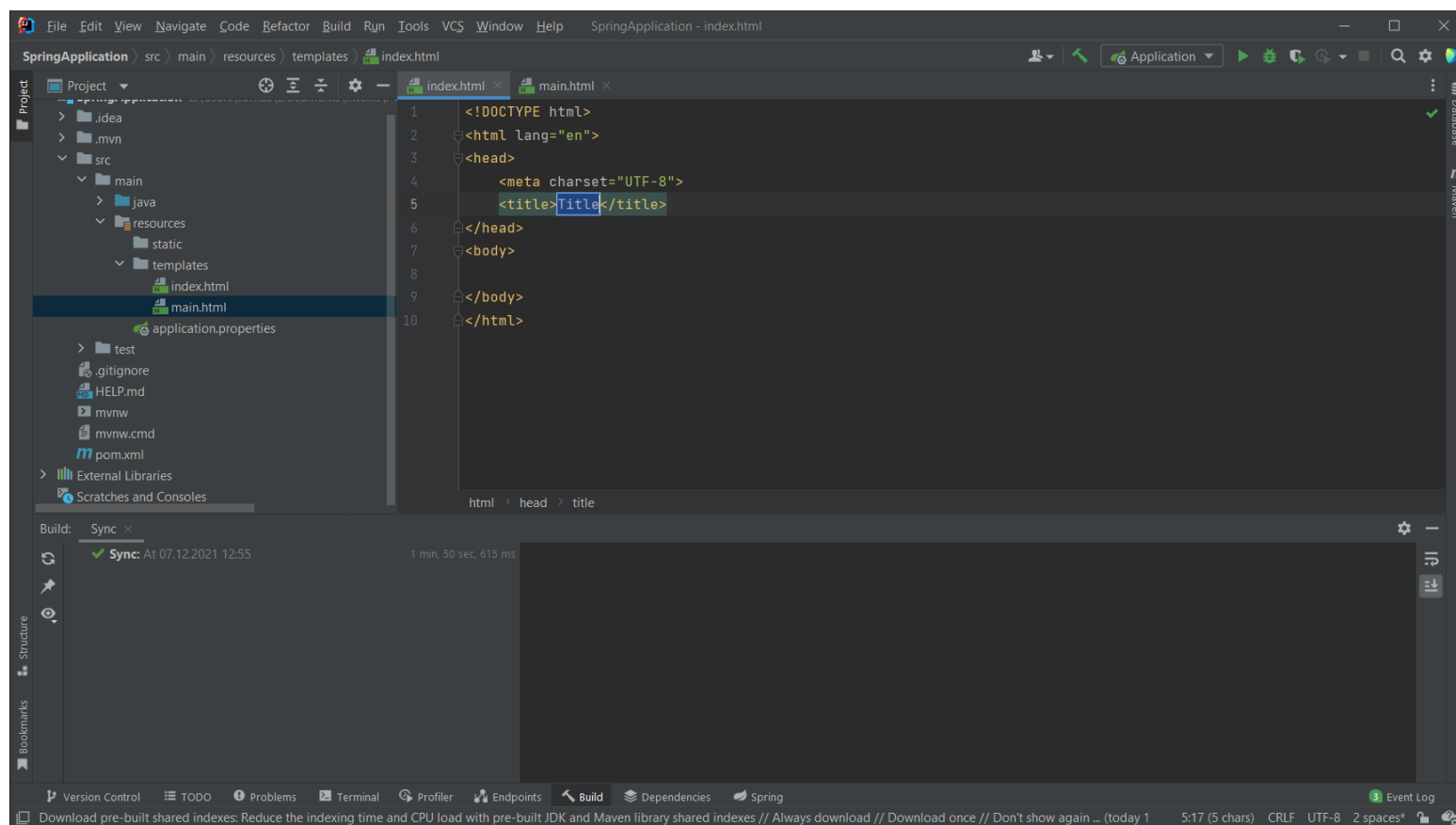
Tworzenie prostej aplikacji webowej ze stroną logowania

We wspomnianej lokalizacji ../templates/ należy utworzyć dwa pliki html (**index** i **main**):



Tworzenie prostej aplikacji webowej ze stroną logowania

Każdy z plików należy uzupełnić o odpowiedni kod (dostępny na kolejnych slajdach).



Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **index.html**, lokalizacja pliku w projekcie: **src/main/resources/templates/index.html**

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-
springsecurity3">
<head>
    <title>Welcome Page</title>
</head>
<body>
<h1>Welcome!</h1>

<p>Click <a th:href="@{/main}">here</a> to see a Main Page.</p>
</body>
</html>
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **index.html** (do skopiowania):

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:th="https://www.thymeleaf.org"
  xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-springsecurity3">
<head>
  <title>Welcome Page</title>
</head>
<body>
<h1>Welcome!</h1>

<p>Click <a th:href="@{/main}">here</a> to see a Main Page.</p>
</body>
</html>
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **main.html**, lokalizacja pliku w projekcie: **src/main/resources/templates/main.html**

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-
springsecurity3">
<head>
    <title>Main Page</title>
</head>
<body>
<h1>Hello world!</h1>
</body>
</html>
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **main.html** (do skopiowania):

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:th="https://www.thymeleaf.org"
  xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-springsecurity3">
<head>
  <title>Main Page</title>
</head>
<body>
<h1>Hello world!</h1>
</body>
</html>
```

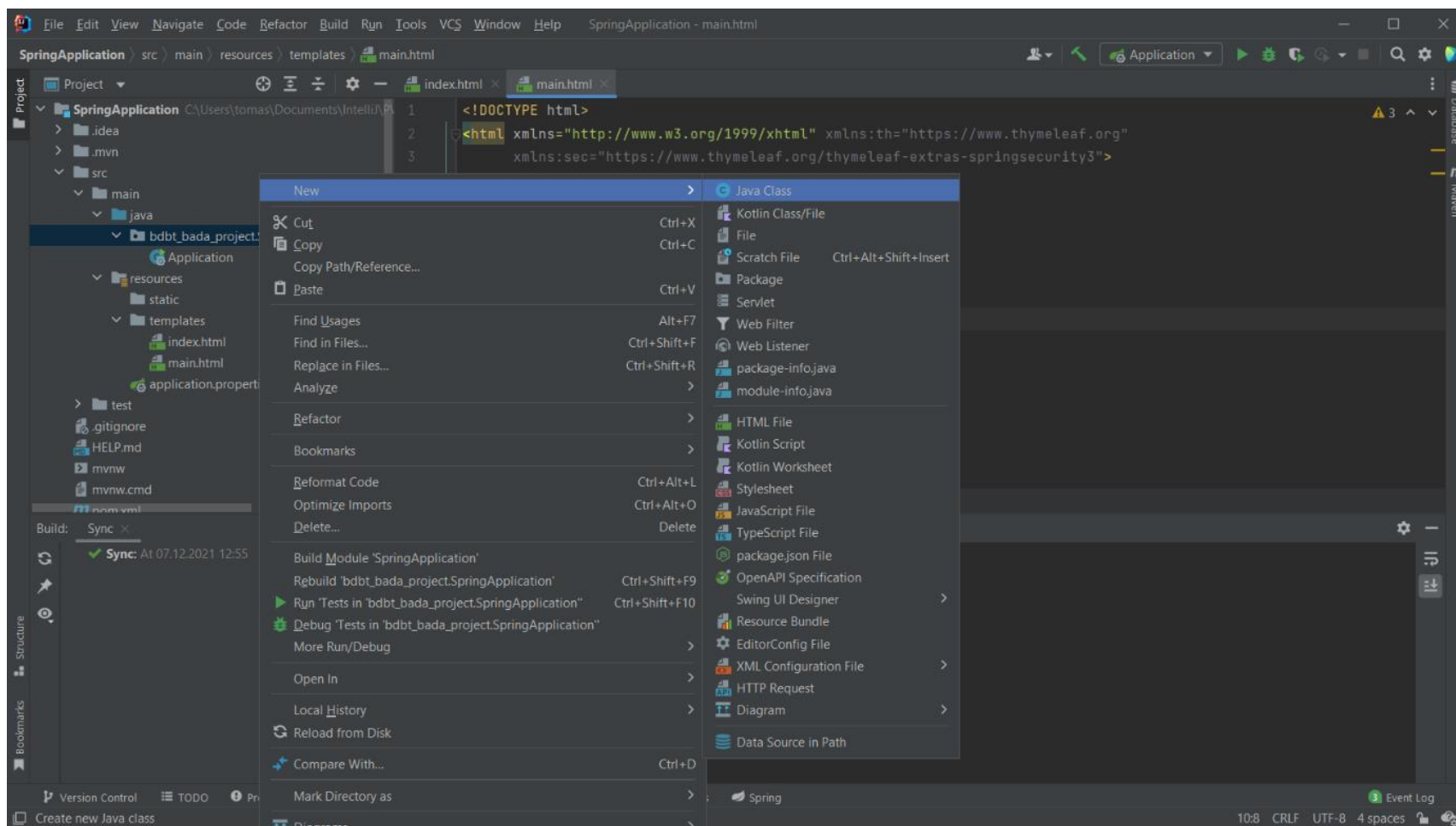
Tworzenie prostej aplikacji webowej ze stroną logowania

Aplikacja webowa bazuje na wzorcu projektowym Spring MVC. Z tego względu należy skonfigurować kontroler widoku (ang. View Controllers), aby móc udostępnić szablony html.

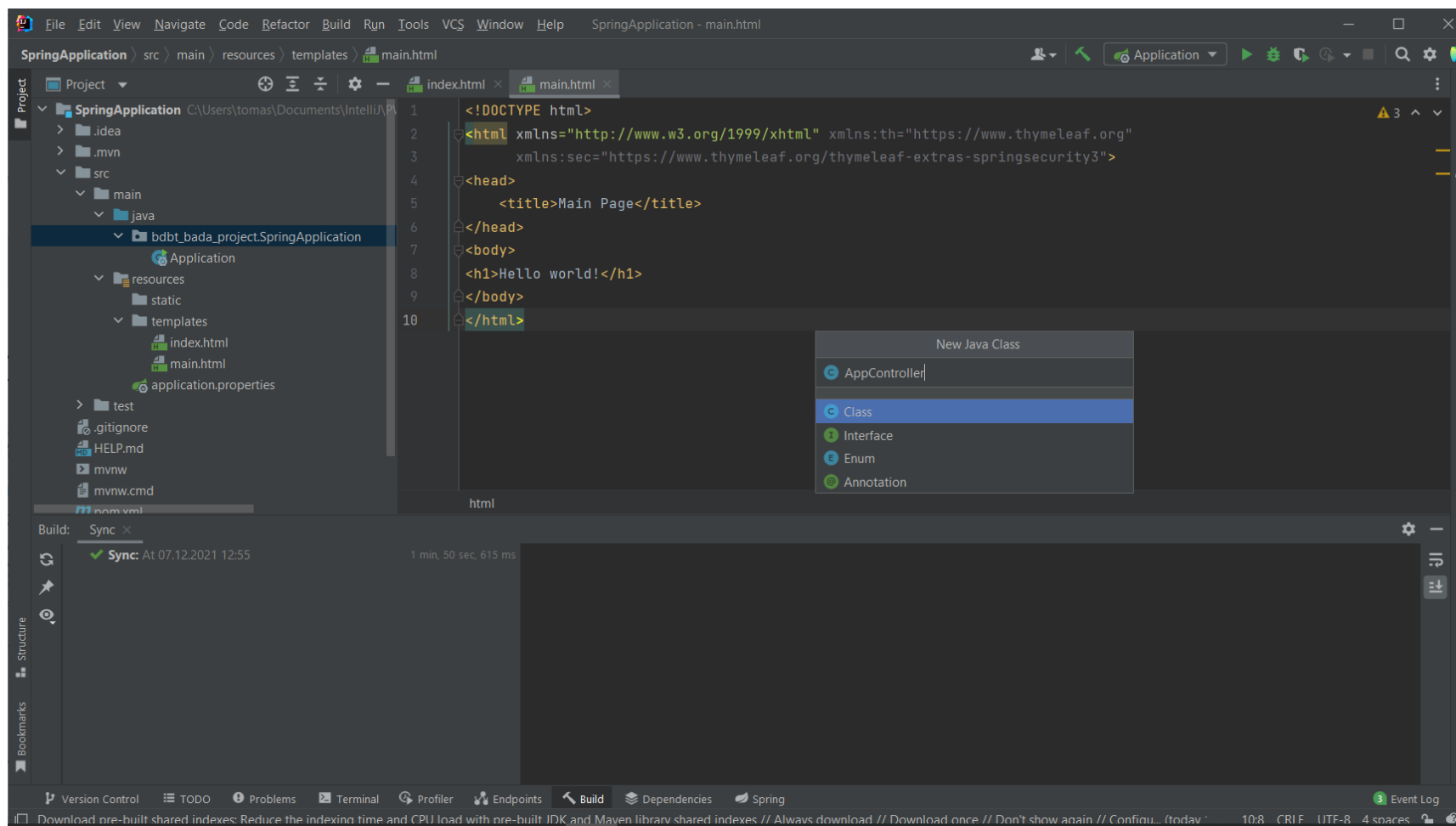
Należy stworzyć plik **AppController.java**

lokalizacja w projekcie: **src/main/java/bdbt_bada_project.SpringApplication/AppController.java**

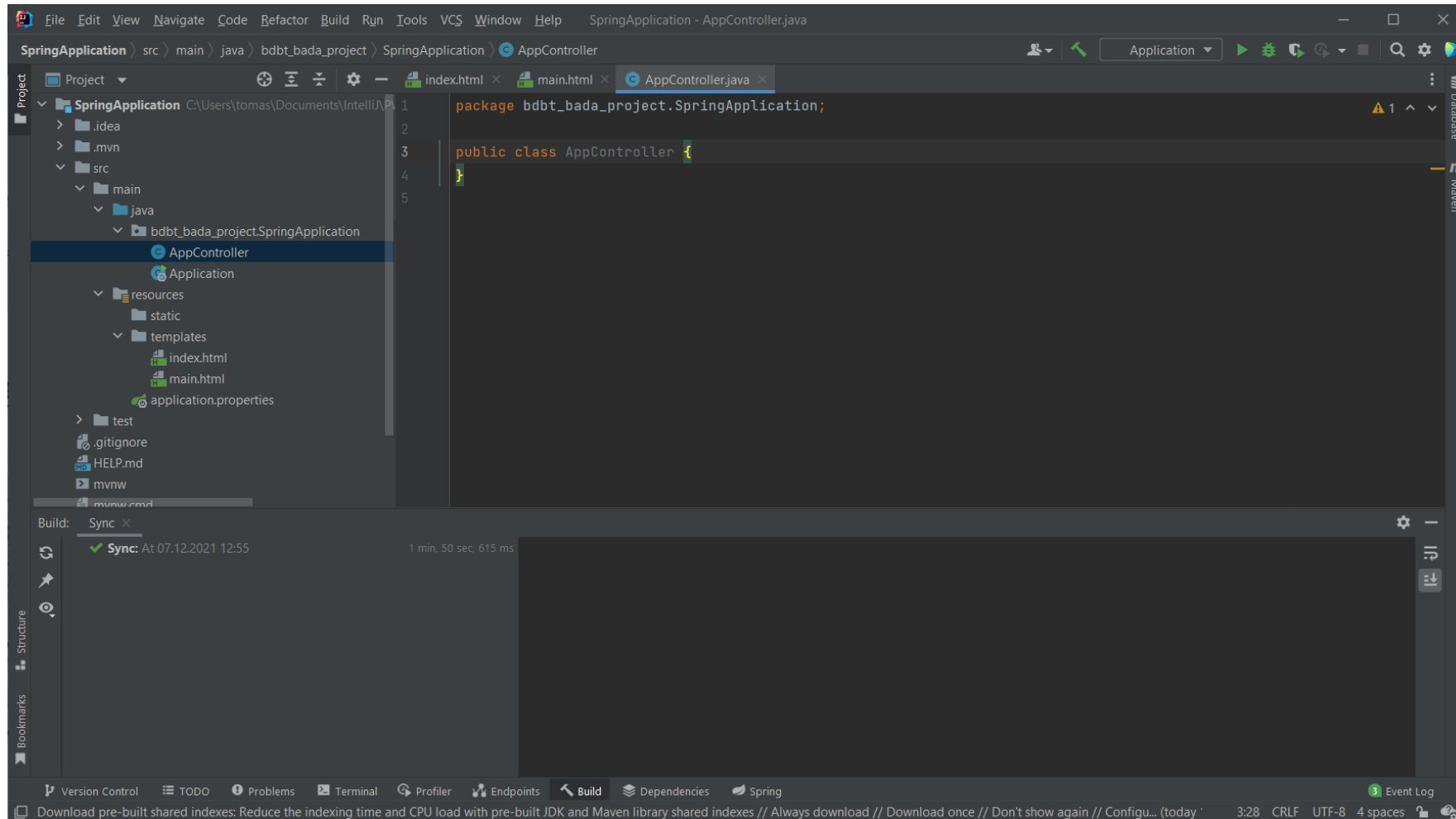
Tworzenie prostej aplikacji webowej ze stroną logowania



Tworzenie prostej aplikacji webowej ze stroną logowania



Tworzenie prostej aplikacji webowej ze stroną logowania



Tworzenie prostej aplikacji webowej ze stroną logowania

Klasa do konfiguracji widoków wg. wzorca MVC (dla Spring). Kod dla pliku **AppController.java**:

```
package bdbt_bada_project.SpringApplication;

import org.springframework.context.annotation.Configuration;
import
org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import
org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class AppController implements WebMvcConfigurer {

    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/index").setViewName("index");
        registry.addViewController("/").setViewName("index");
        registry.addViewController("/main").setViewName("main");
        registry.addViewController("/login").setViewName("login");
    }
}
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **AppController.java** (do skopiowania):

```
package bdbt_bada_project.SpringApplication;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

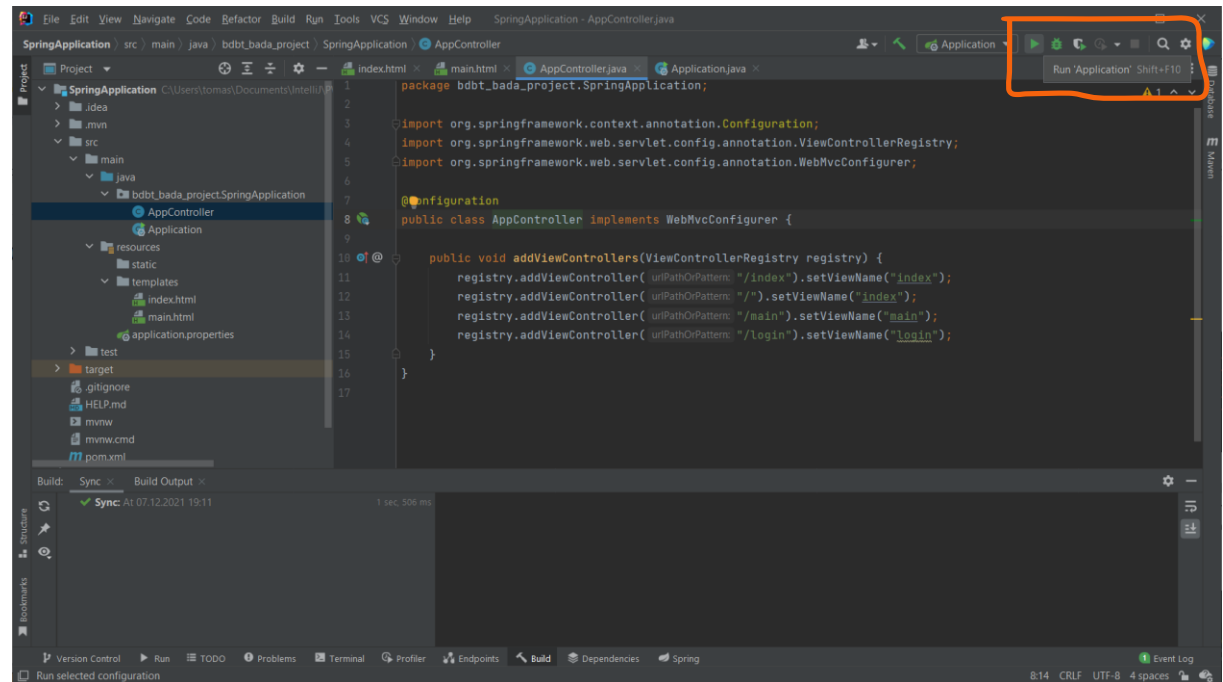
@Configuration
public class AppController implements WebMvcConfigurer {

    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/index").setViewName("index");
        registry.addViewController("/").setViewName("index");
        registry.addViewController("/main").setViewName("main");
        registry.addViewController("/login").setViewName("login");
    }
}
```

Tworzenie prostej aplikacji webowej ze stroną logowania

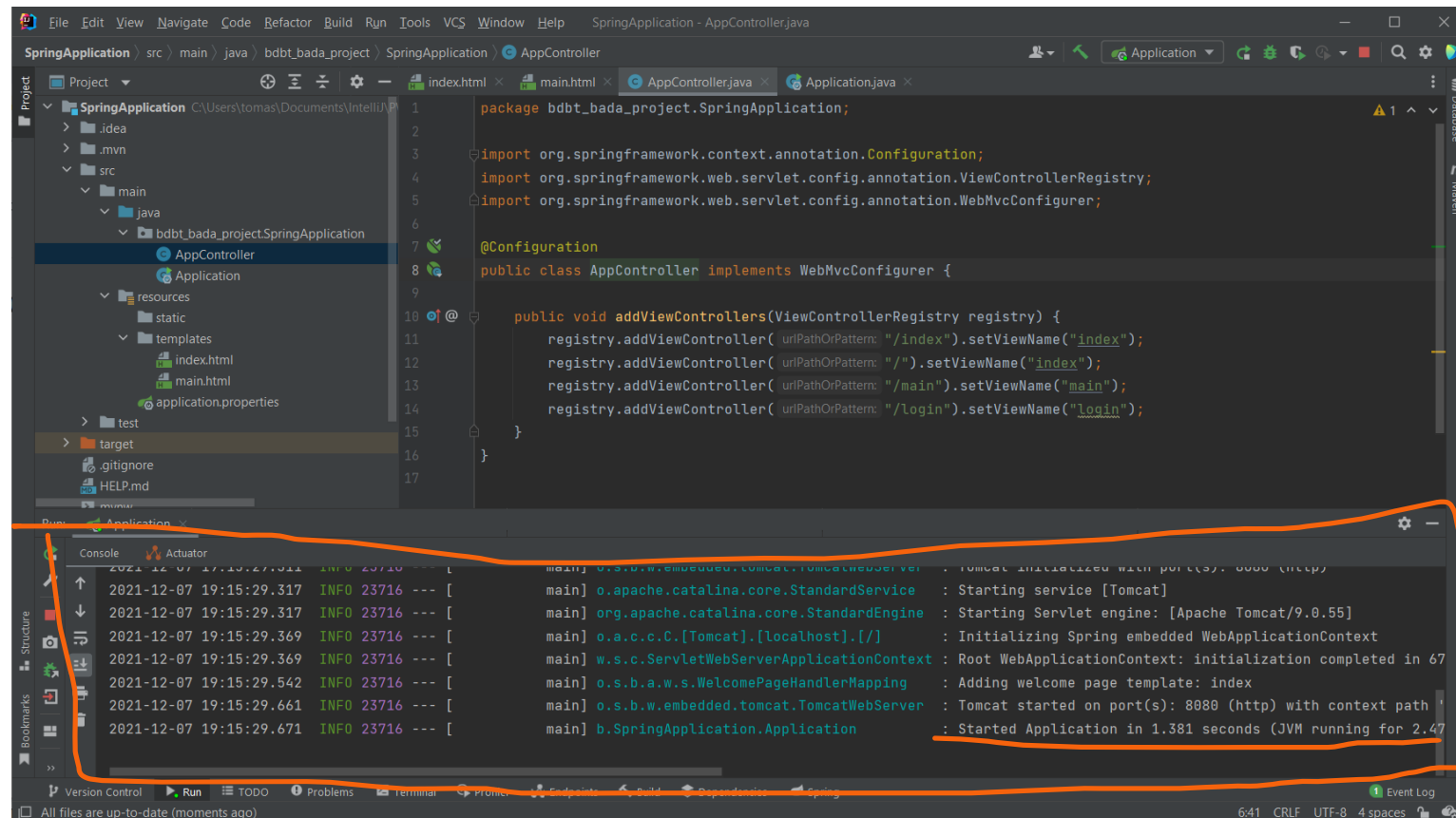
Metoda **addViewControllers()** dodaje w tym przypadku cztery kontrolery widoku. Dwa odwołują się do **index.html**, tj. strony przed zalogowaniem, jedna do strony widocznej po zalogowaniu **main.html**. Ostatni widok jest dla **login.html**, czyli widoku dostępnego w trakcie logowania.

W tym momencie można uruchomić aplikację, klikając w przycisk **Run 'Application'**



Tworzenie prostej aplikacji webowej ze stroną logowania

Poprawnie napisany kod wyświetli w rezultacie w Console następujące komunikaty:



```
package bdbt_bada_project.SpringApplication;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class AppController implements WebMvcConfigurer {

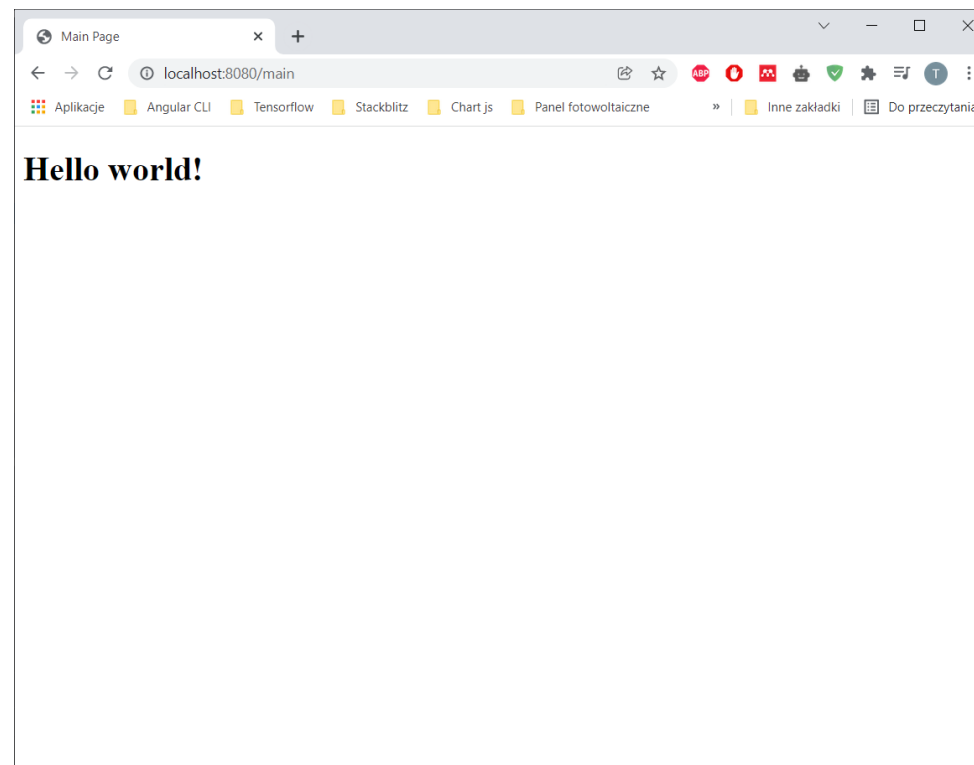
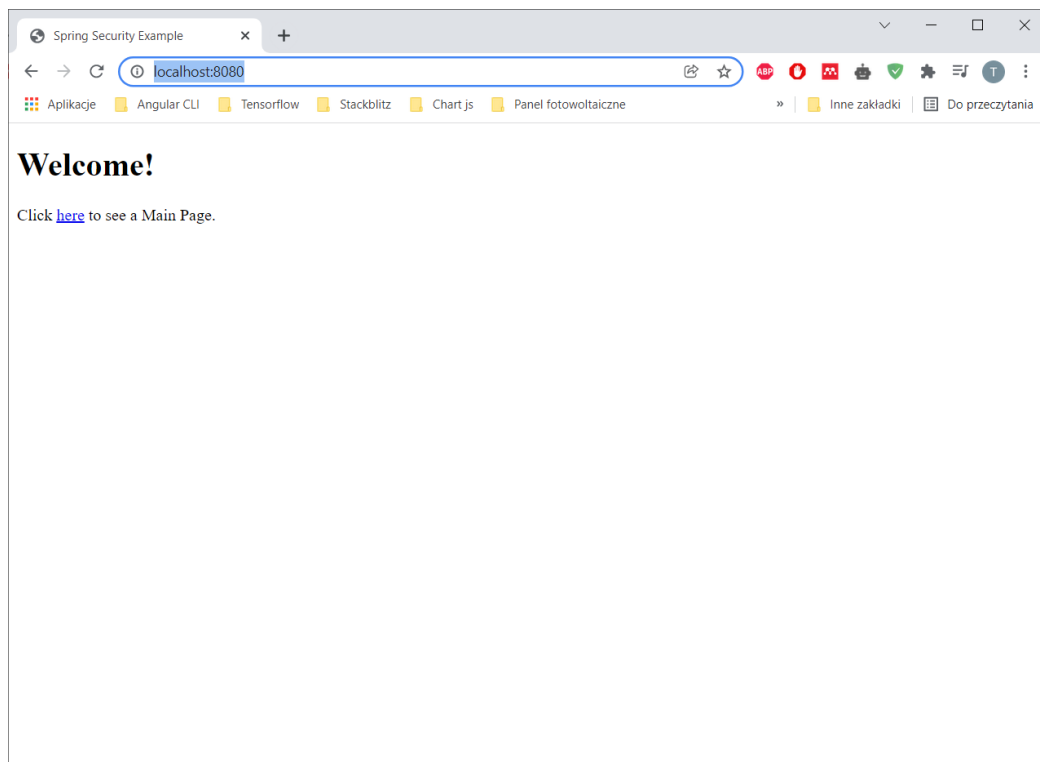
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/index").setViewName("index");
        registry.addViewController("/").setViewName("index");
        registry.addViewController("/main").setViewName("main");
        registry.addViewController("/login").setViewName("login");
    }
}
```

Console output:

```
2021-12-07 19:15:29.311 INFO 23716 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2021-12-07 19:15:29.317 INFO 23716 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2021-12-07 19:15:29.317 INFO 23716 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.55]
2021-12-07 19:15:29.369 INFO 23716 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2021-12-07 19:15:29.369 INFO 23716 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 67
2021-12-07 19:15:29.542 INFO 23716 --- [main] o.s.b.a.w.s.WelcomePageHandlerMapping : Adding welcome page template: index
2021-12-07 19:15:29.661 INFO 23716 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path
2021-12-07 19:15:29.671 INFO 23716 --- [main] b.SpringApplication.Application : Started Application in 1.381 seconds (JVM running for 2.47)
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Aplikacja jest dostępna w przeglądarce pod adresem: <http://localhost:8080/> oraz po przejściu na stronę główną (Hello world!):



2

STRONA LOGOWANIA

Tworzenie prostej aplikacji webowej ze stroną logowania

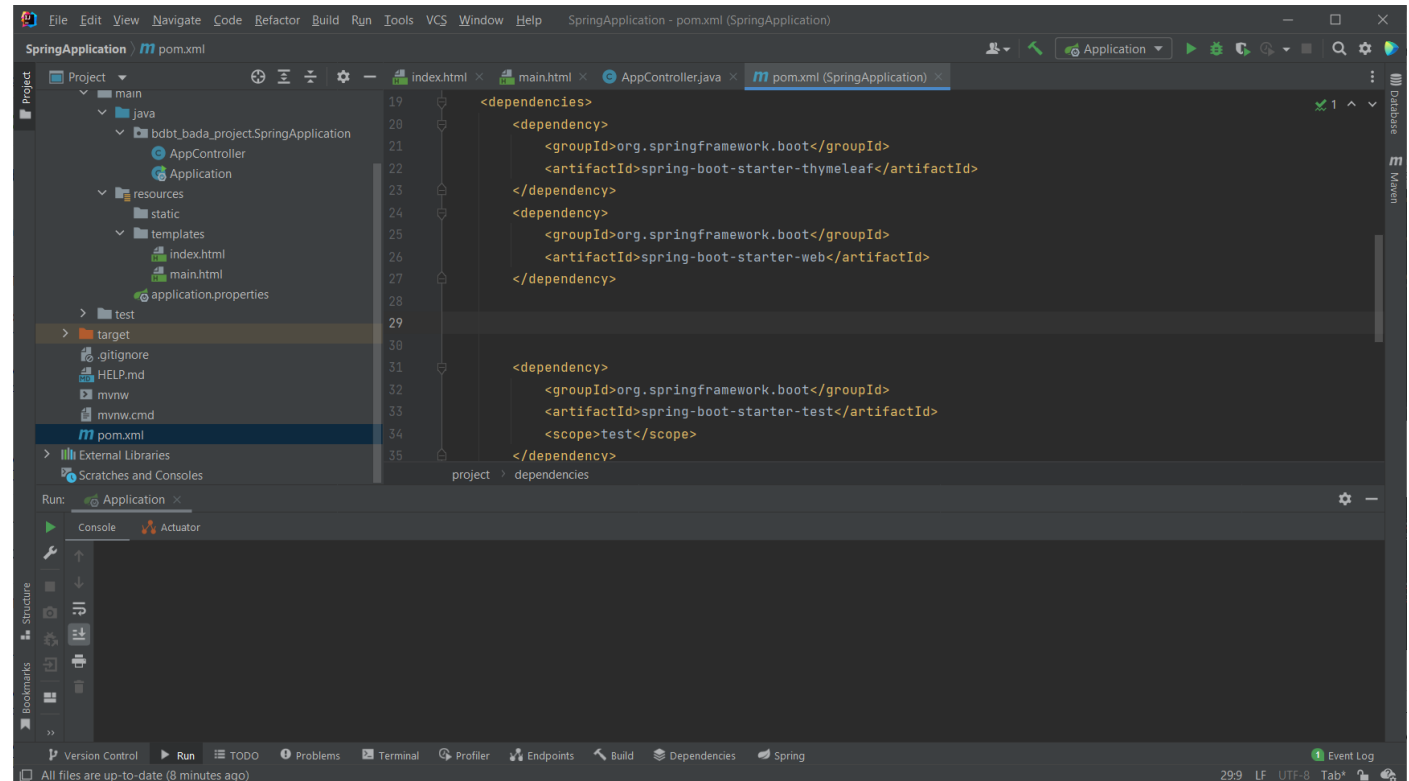
Następnym etapem tworzonej aplikacji, jest uniemożliwienie przeglądania strony głównej (main.html) przez osoby niezalogowane. Aby ją zobaczyć konieczne będzie przejście przez proces logowania.

Aby zabezpieczyć aplikację konieczne jest użycie specjalnych mechanizmów. W tym celu do zabezpieczenia zostanie wykorzystana biblioteka Spring Security.

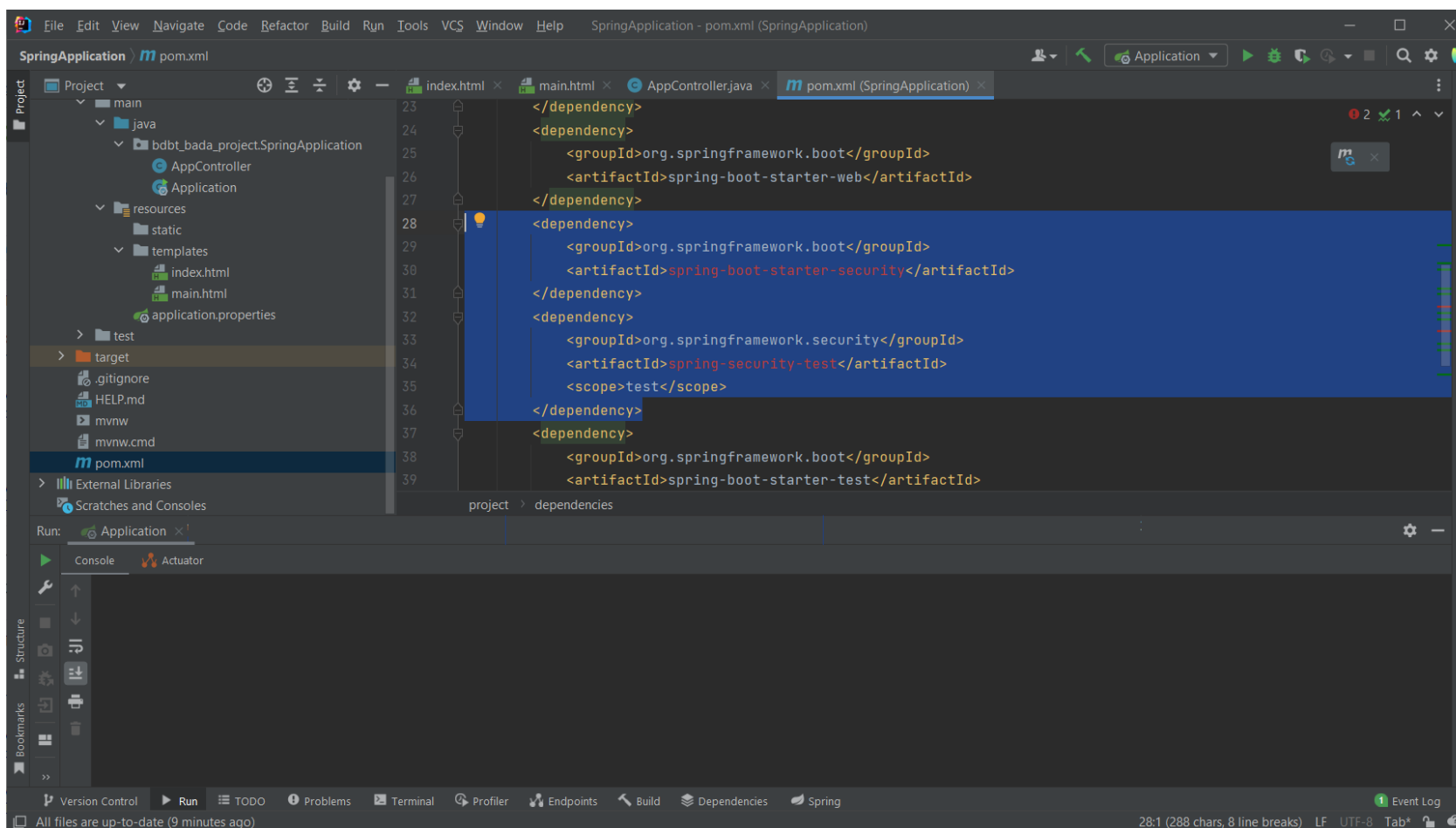
Tworzenie prostej aplikacji webowej ze stroną logowania

W omawianym przykładzie stworzony projekt wykorzystuje narzędzie Maven. Aby zabezpieczyć aplikację należy:

a) Dodać zależności w pliku pom.xml



Tworzenie prostej aplikacji webowej ze stroną logowania



Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **pom.xml**:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

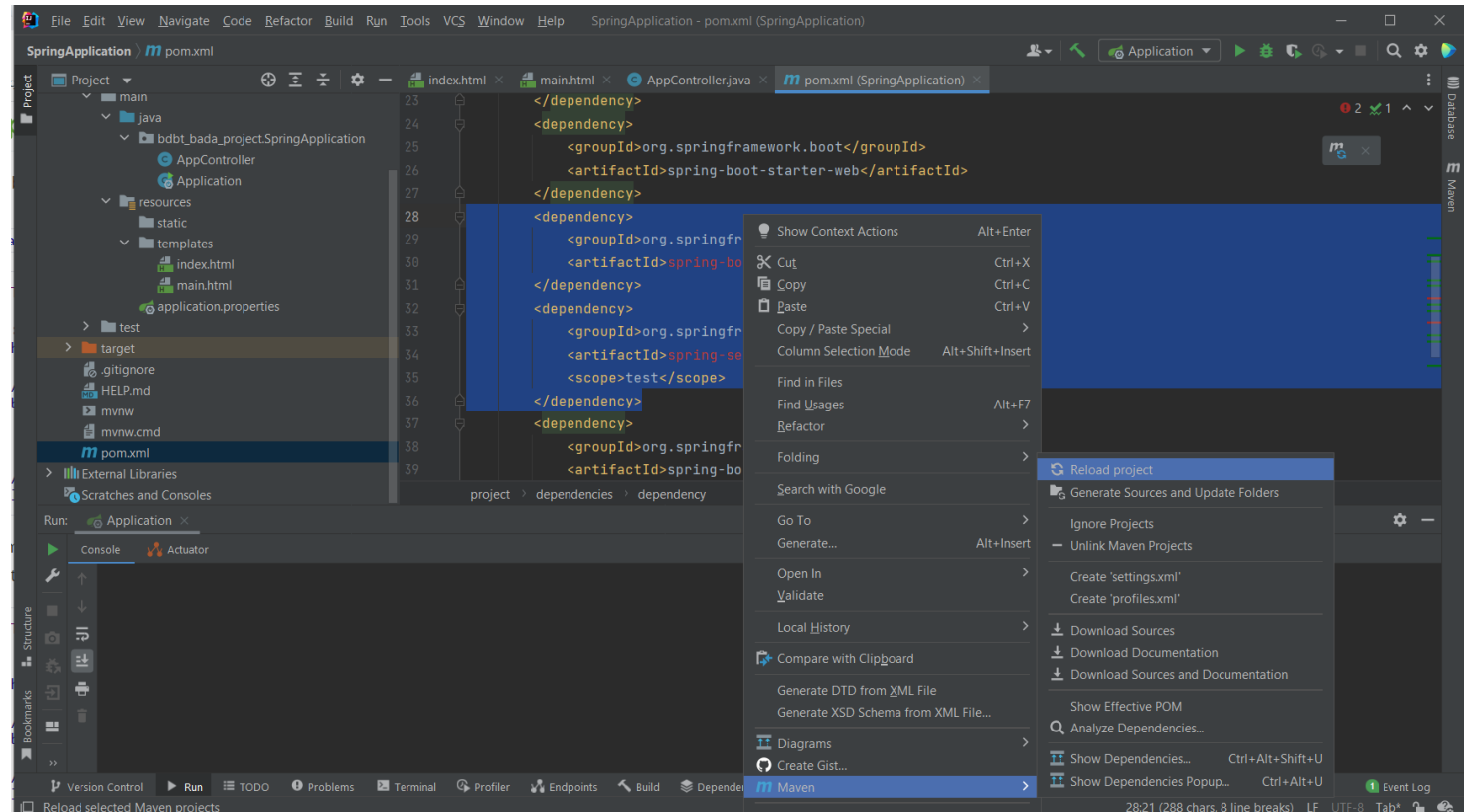
Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **pom.xml** (do skopiowania):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

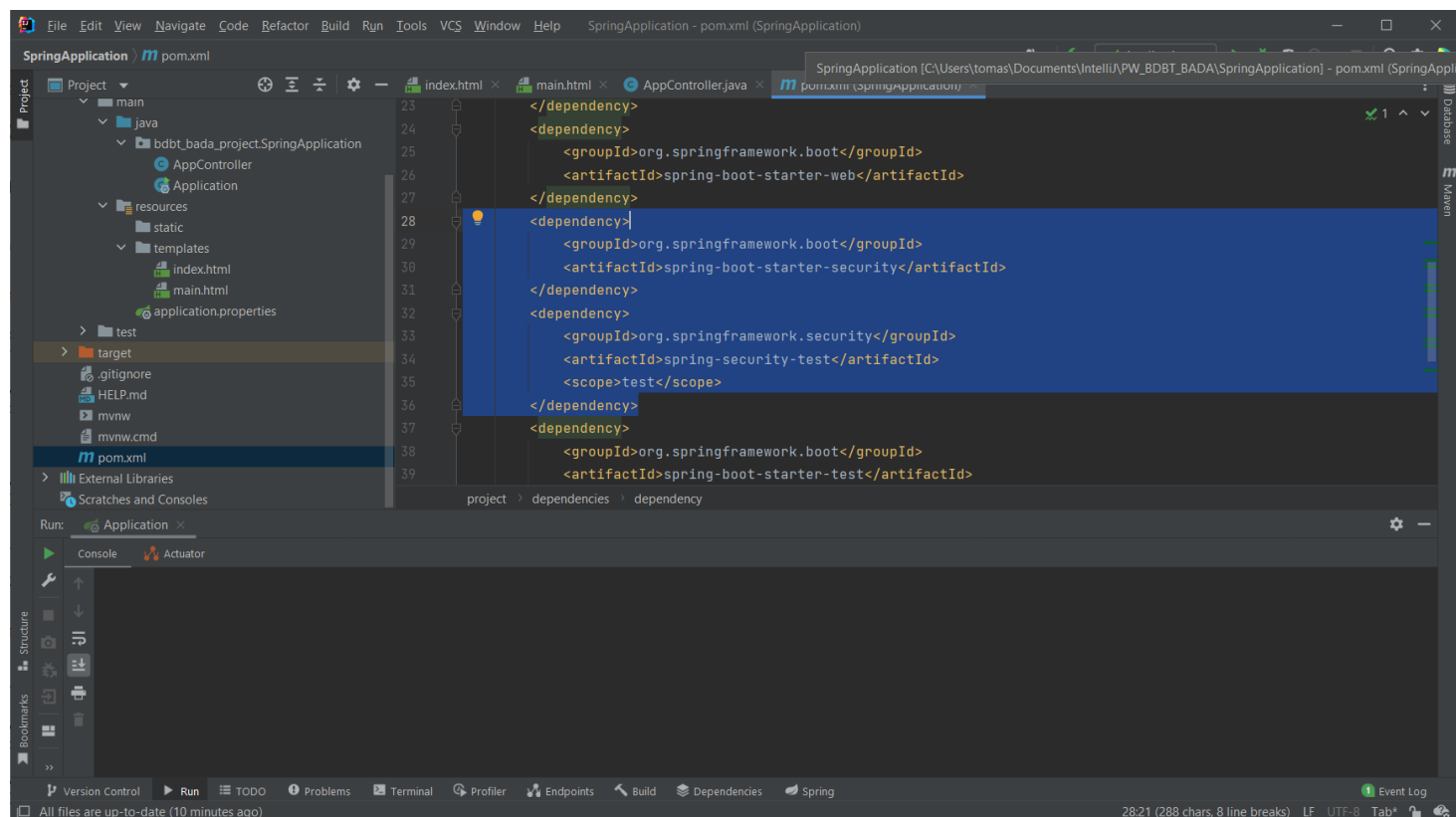
Tworzenie prostej aplikacji webowej ze stroną logowania

Aby zainstalować zależności (odświeżyć projekt) należy kliknąć prawym klawiszem myszki w pole kodu w pliku pom.xml, lub w nazwę projektu w drzewie (SpringApplication) lub w nazwę pliku pom.xml i z pozycji Maven wybrać opcję Reload Project.



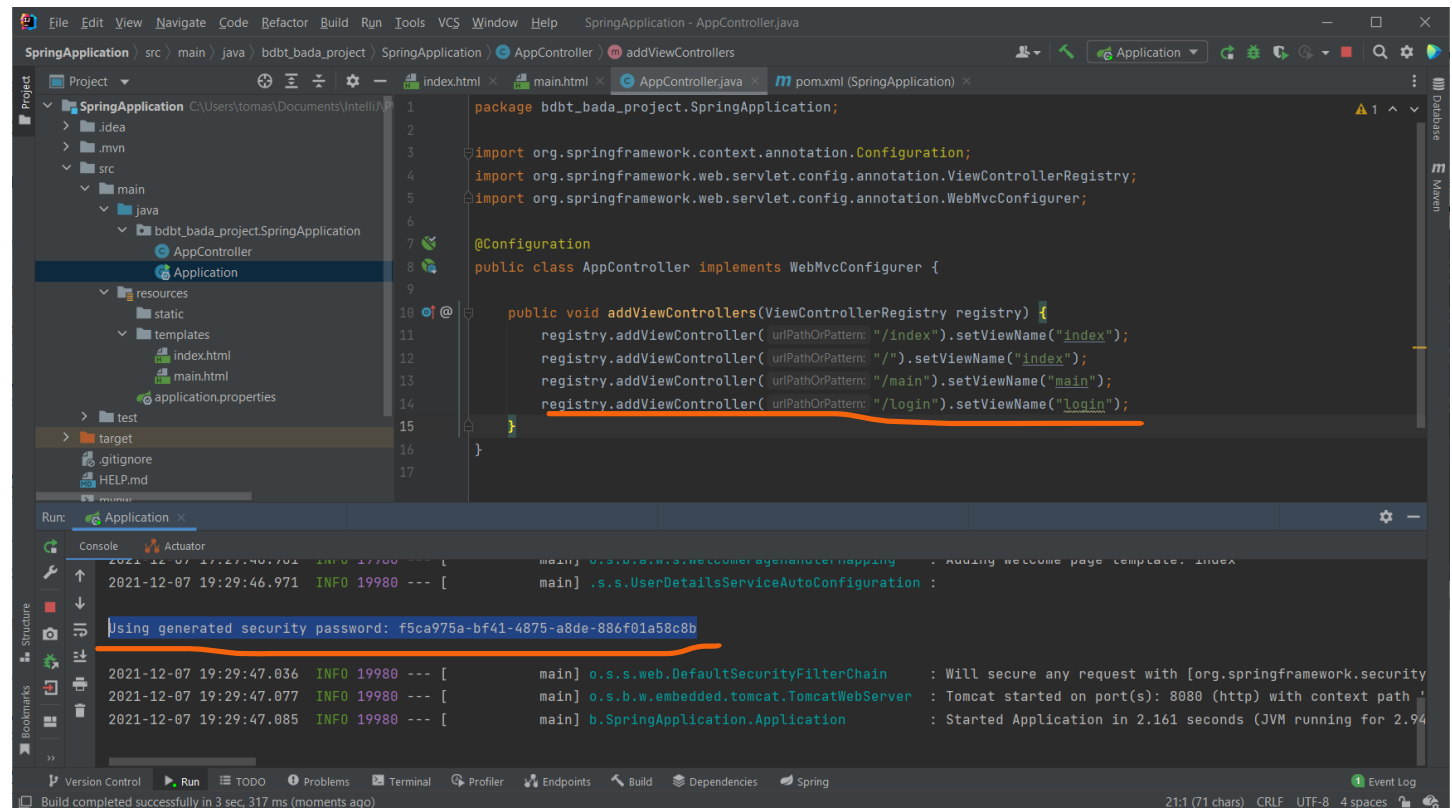
Tworzenie prostej aplikacji webowej ze stroną logowania

Wszystkie brakujące/niezainstalowane/nowo dodane zależności zostaną zainstalowane, a w pliku pom.xml znikną czerwone podkreślniki.



Tworzenie prostej aplikacji webowej ze stroną logowania

Zainstalowanie bibliotek odpowiedzialnych za zabezpieczenie spowoduje, że po uruchomieniu strony i wybraniu adresu <http://localhost:8080/> nastąpi automatyczne przekierowanie na stronę logowania (login).



```
package bdbt_bada_project.SpringApplication;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class AppController implements WebMvcConfigurer {

    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController( uriPathOrPattern: "/" ).setViewName("index");
        registry.addViewController( uriPathOrPattern: "/" ).setViewName("index");
        registry.addViewController( uriPathOrPattern: "/main" ).setViewName("main");
        registry.addViewController( uriPathOrPattern: "/login" ).setViewName("login");
    }
}
```

Run: Application

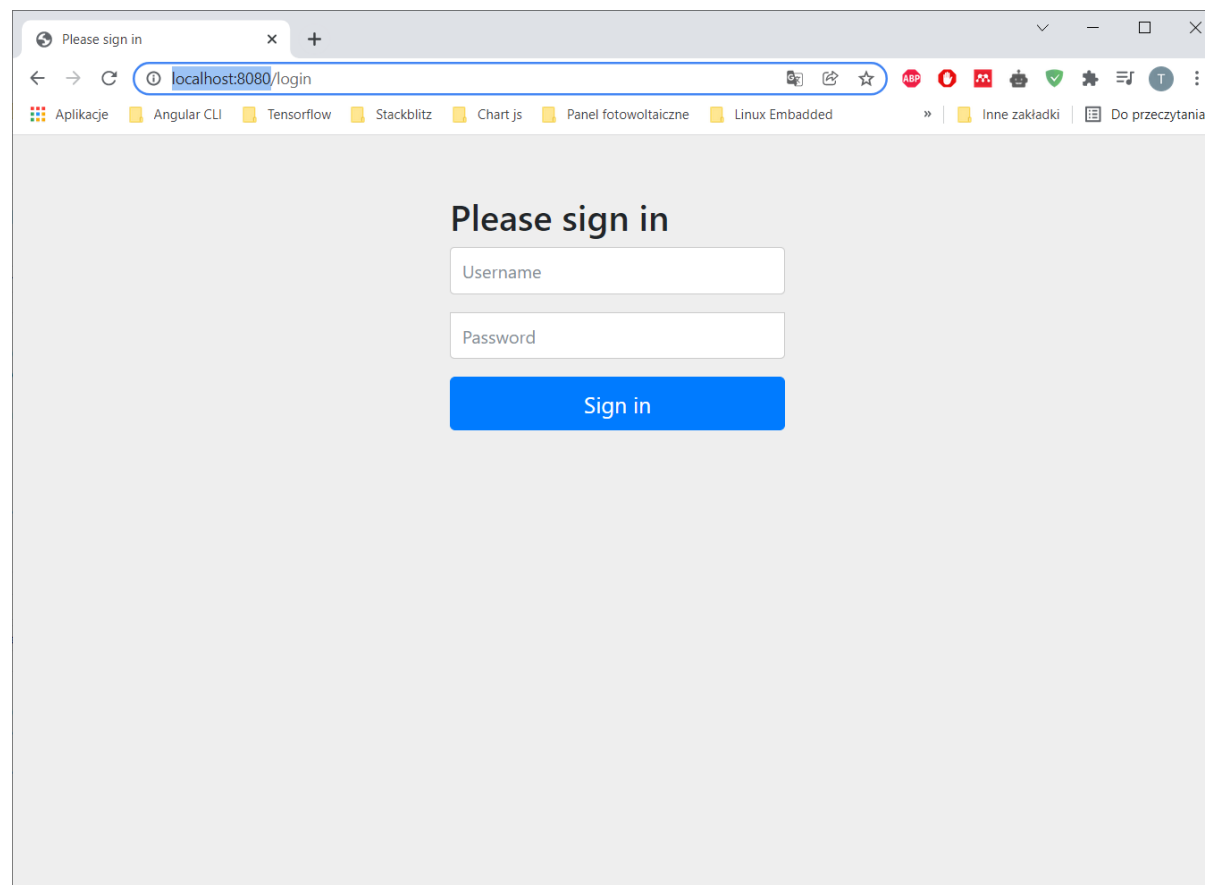
Console

```
2021-12-07 19:29:46.971 INFO 19980 --- [main] o.s.b.w.s.welcome.WelcomeControllerMapping : Adding welcome page template: index
2021-12-07 19:29:46.971 INFO 19980 --- [main] o.s.s.UserDetailsServiceAutoConfiguration :
Using generated security password: f5ca975a-bf41-4875-a8de-886f01a58c8b
2021-12-07 19:29:47.036 INFO 19980 --- [main] o.s.s.web.DefaultSecurityFilterChain : Will secure any request with [org.springframework.security
2021-12-07 19:29:47.077 INFO 19980 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path
2021-12-07 19:29:47.085 INFO 19980 --- [main] b.SpringApplication.Application : Started Application in 2.161 seconds (JVM running for 2.94
```

Build completed successfully in 3 sec, 317 ms (moments ago)

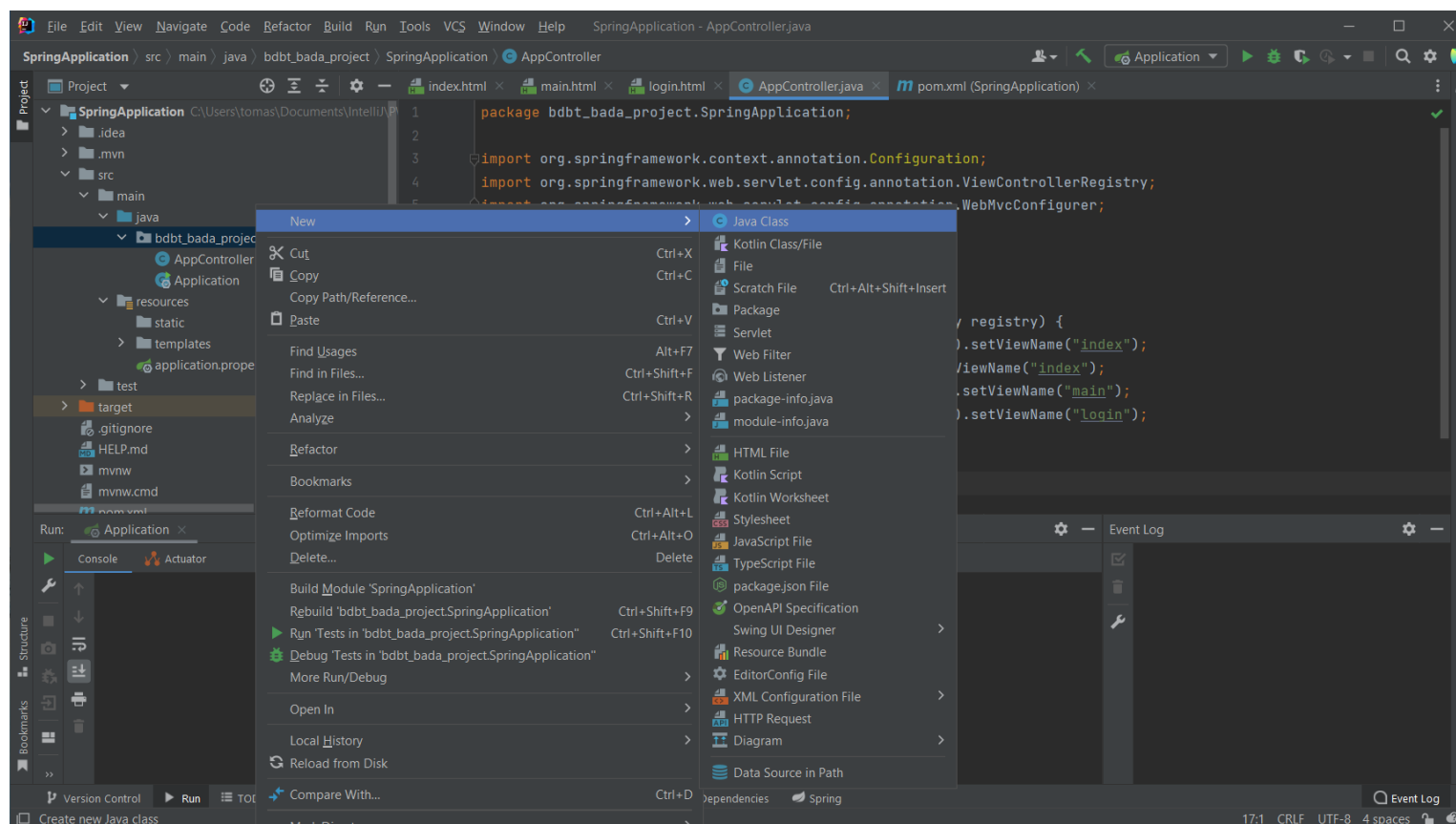
Tworzenie prostej aplikacji webowej ze stroną logowania

Jak widać, jest to strona logowania, która wyświetla się automatycznie nawet, bez posiadania zdefiniowanej strony w szablonach z plikami html.

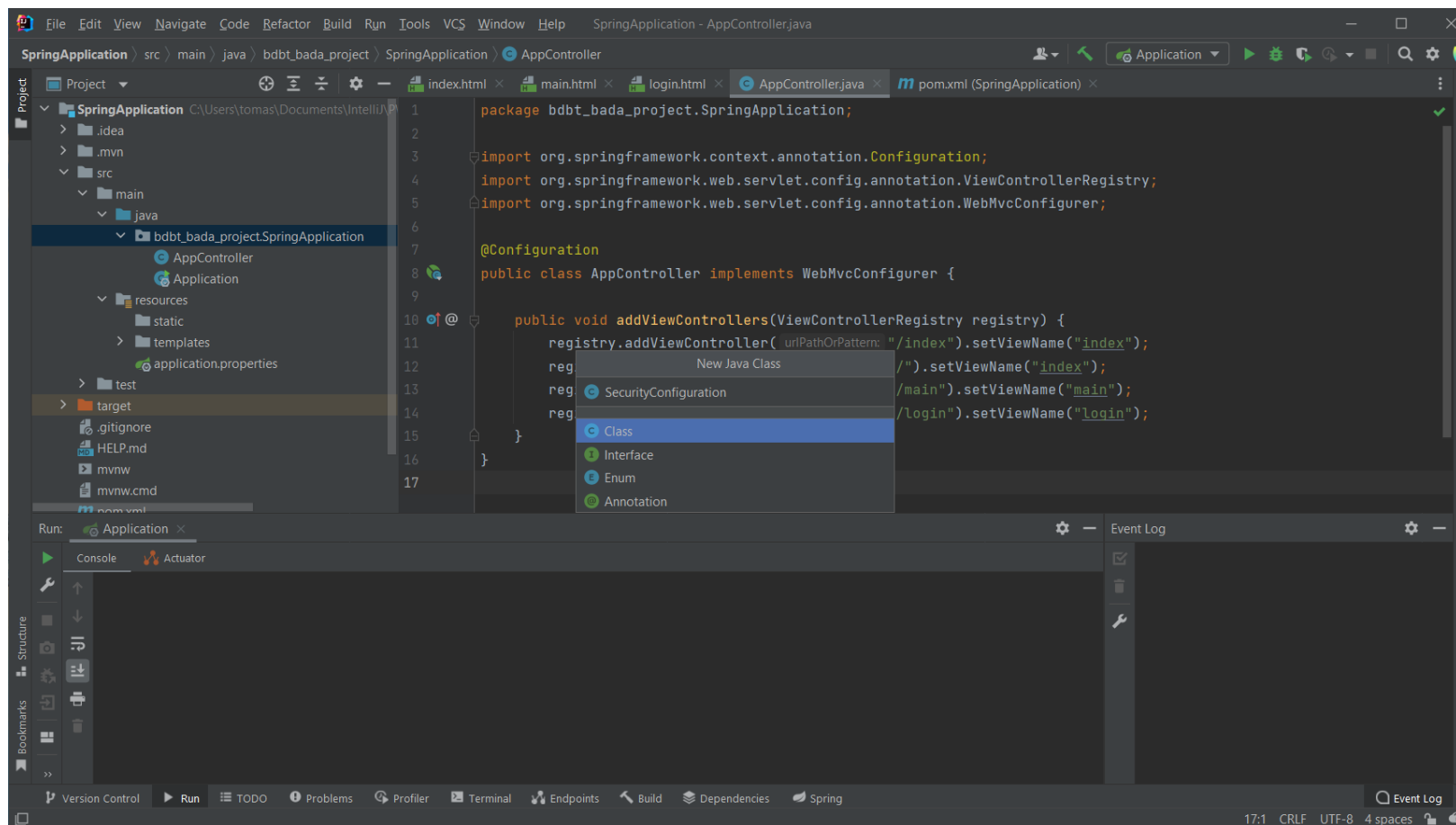


Tworzenie prostej aplikacji webowej ze stroną logowania

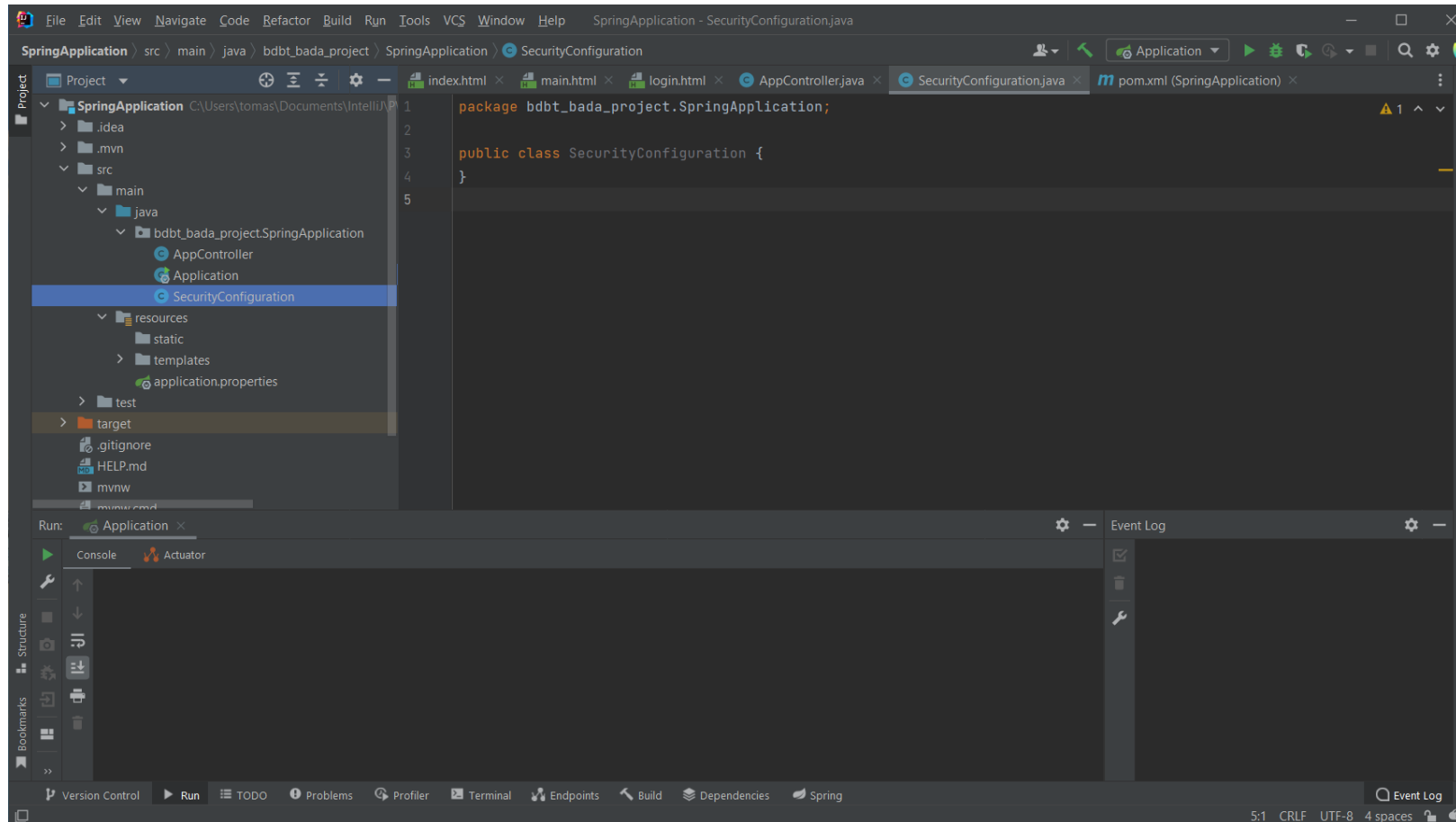
Aby zapewnić pełną obsługę ze stroną logowania, należy dodać specjalnie przeznaczoną do tego klasę.



Tworzenie prostej aplikacji webowej ze stroną logowania



Tworzenie prostej aplikacji webowej ze stroną logowania



Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **SecurityConfiguration.java** (cz. 1):

```
package bdbt_bada_project.SpringApplication;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter;
import org.springframework.security.crypto.password.NoOpPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
@EnableWebSecurity
public class SecurityConfiguration extends WebSecurityConfigurerAdapter {
    @Override
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **SecurityConfiguration.java** (cz. 2):

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
        auth.inMemoryAuthentication()
            .withUser("user")
            .password("user")
            .roles("USER")
            .and()
            .withUser("admin")
            .password("admin")
            .roles("ADMIN");
    }

    @Bean
    public PasswordEncoder getPasswordEncoder() { return
NoOpPasswordEncoder.getInstance();
    }
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **SecurityConfiguration.java** (cz. 3):

```
public PasswordEncoder getPasswordEncoder() { return  
NoOpPasswordEncoder.getInstance();  
}  
  
@Override  
protected void configure(HttpSecurity http) throws Exception {  
    http  
        .authorizeRequests()  
        .antMatchers("/", "/index").permitAll()  
        .anyRequest().authenticated()  
        .and()  
        .formLogin()  
        .loginPage("/login")  
        .permitAll()  
        .and()  
        .logout()  
        .permitAll();  
}  
}
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **SecurityConfiguration.java** (do skopiowania cz. 1):

```
package bdbt_bada_project.SpringApplication;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter;
import org.springframework.security.crypto.password.NoOpPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **SecurityConfiguration.java** (do skopiowania cz. 2):

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.inMemoryAuthentication()
            .withUser("user")
            .password("user")
            .roles("USER")
            .and()
            .withUser("admin")
            .password("admin")
            .roles("ADMIN");
    }
}
```


Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **SecurityConfiguration.java** (do skopiowania cz. 3):

```
@Bean  
public PasswordEncoder getPasswordEncoder() { return NoOpPasswordEncoder.getInstance();  
}
```

Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **SecurityConfiguration.java** (do skopiowania cz. 4):

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
        .antMatchers("/", "/index").permitAll()
        .anyRequest().authenticated()
        .and()
        .formLogin()
        .loginPage("/login")
        .permitAll()
        .and()
        .logout()
        .permitAll();
}
```

Tworzenie prostej aplikacji webowej ze stroną logowania

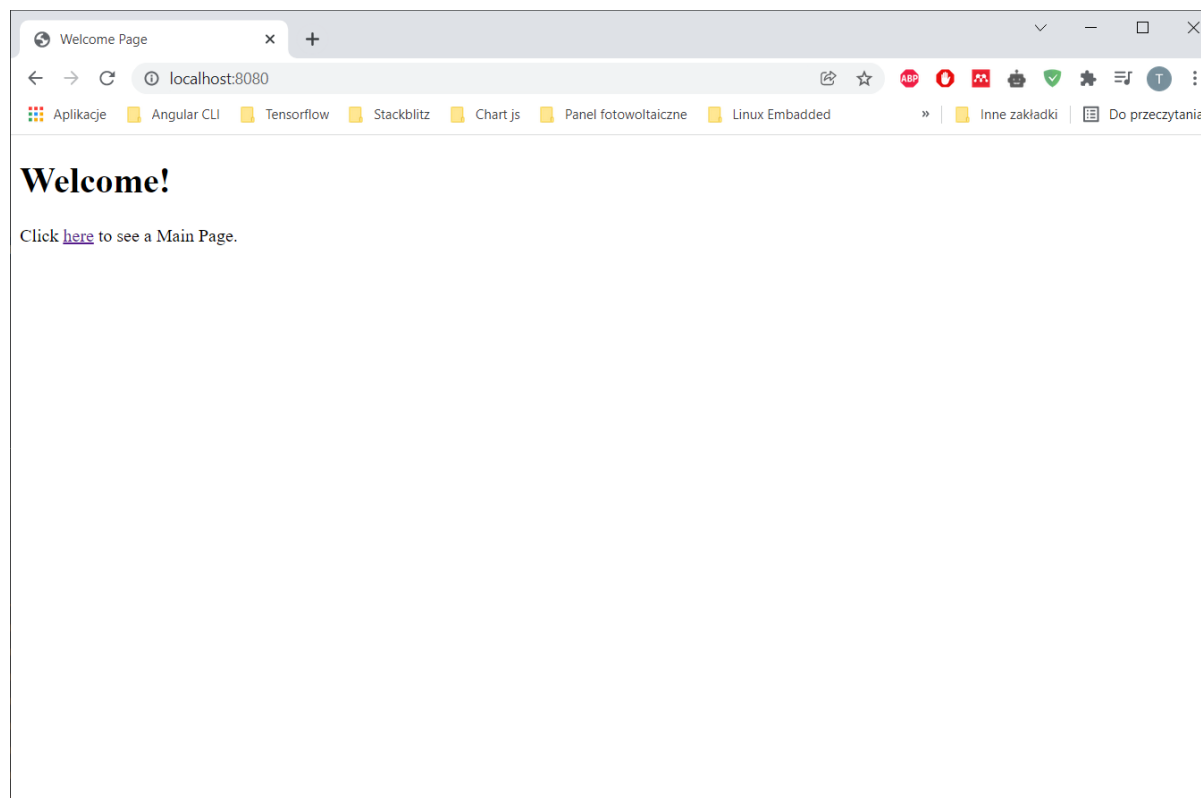
Klasa `SecurityConfiguration` jest opatrzona adnotacją `@EnableWebSecurity`, aby włączyć obsługę zabezpieczeń Spring Security i zapewnić ich integrację dla Spring MVC. Klasa jest również rozszerzona o `WebSecurityConfigurerAdapter` aby zwiększyć możliwości konfiguracyjne zabezpieczeń.

Metoda `configure(HttpSecurity)` definiuje, które ścieżki URL powinny być zabezpieczone, a które nie. W tym przypadku ścieżki `/` oraz `/index` są skonfigurowane tak, aby nie wymagały uwierzytelniania. Pozostałe muszą być uwierzytelnione.

Gdy użytkownik poprawnie się zaloguje, to zostanie przekierowany na żadaną stronę, która wymaga uwierzytelnienia. Istnieje strona `/login` (przez `.loginPage`), która jest widoczna dla każdego.

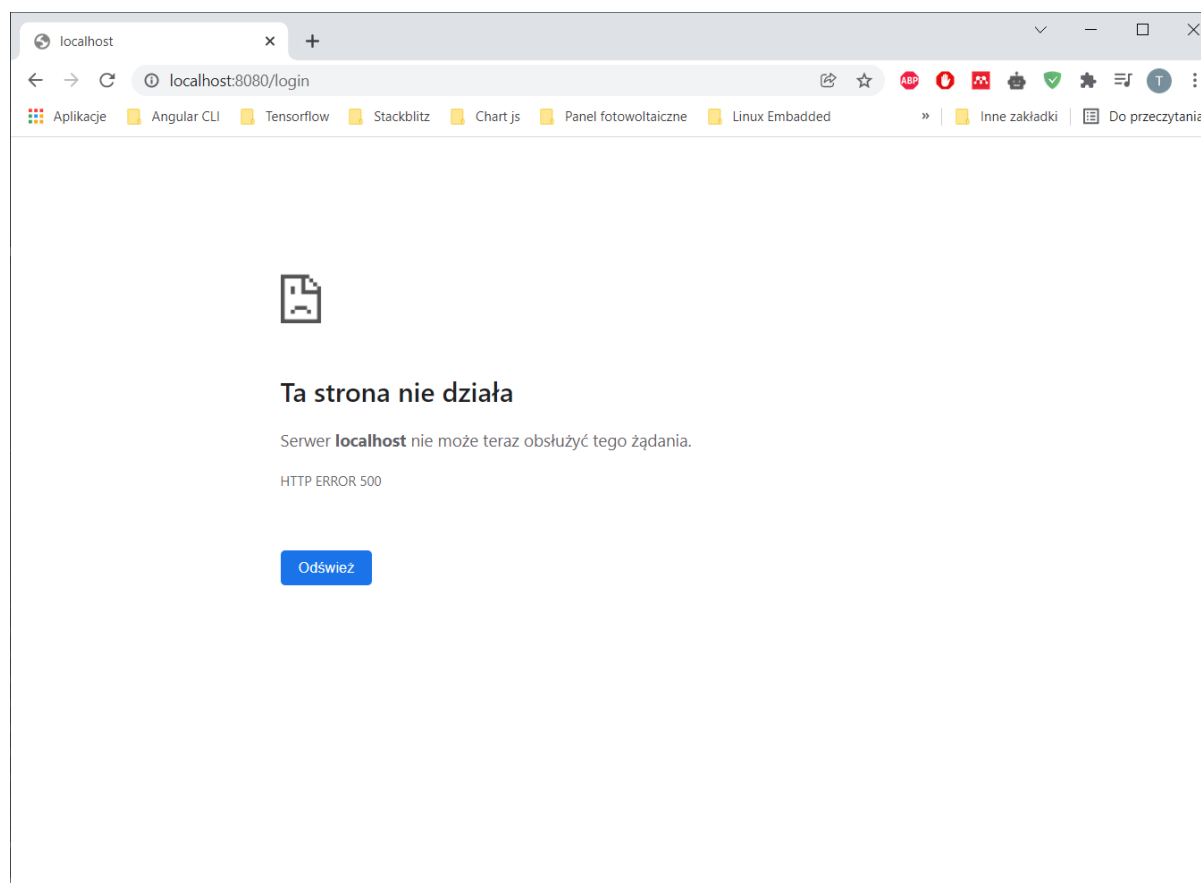
Tworzenie prostej aplikacji webowej ze stroną logowania

Dodanie tej klasy spowoduje, że po uruchomieniu aplikacji i przejściu na adres <http://localhost:8080/> zostanie wyświetlona początkowa strona z pliku index.html:



Tworzenie prostej aplikacji webowej ze stroną logowania

Natomiast próba przejścia do strony głównej main.html spowoduje błąd (ponieważ nie został utworzony plik login.html.):



Tworzenie prostej aplikacji webowej ze stroną logowania

Kolejnym etapem jest stworzenie pliku login.html do strony logowania. Lokalizacja pliku jest standardowa: **src/main/resources/templates/login.html**. Kod dla pliku **login.html**:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-
springsecurity3">
<head>
    <title>Authentication Page</title>
</head>
<body>
<div th:if="{param.error}">
    Invalid username and password.
</div>
<div th:if="{param.logout}">
    You have been logged out.
</div>
<form th:action="{@{/login}}" method="post">
    <div><label> User Name : <input type="text" name="username"/>
</label></div>
    <div><label> Password: <input type="password" name="password"/>
</label></div>
    <div><input type="submit" value="Sign In"/></div>
</form>
</body>
</html>
```

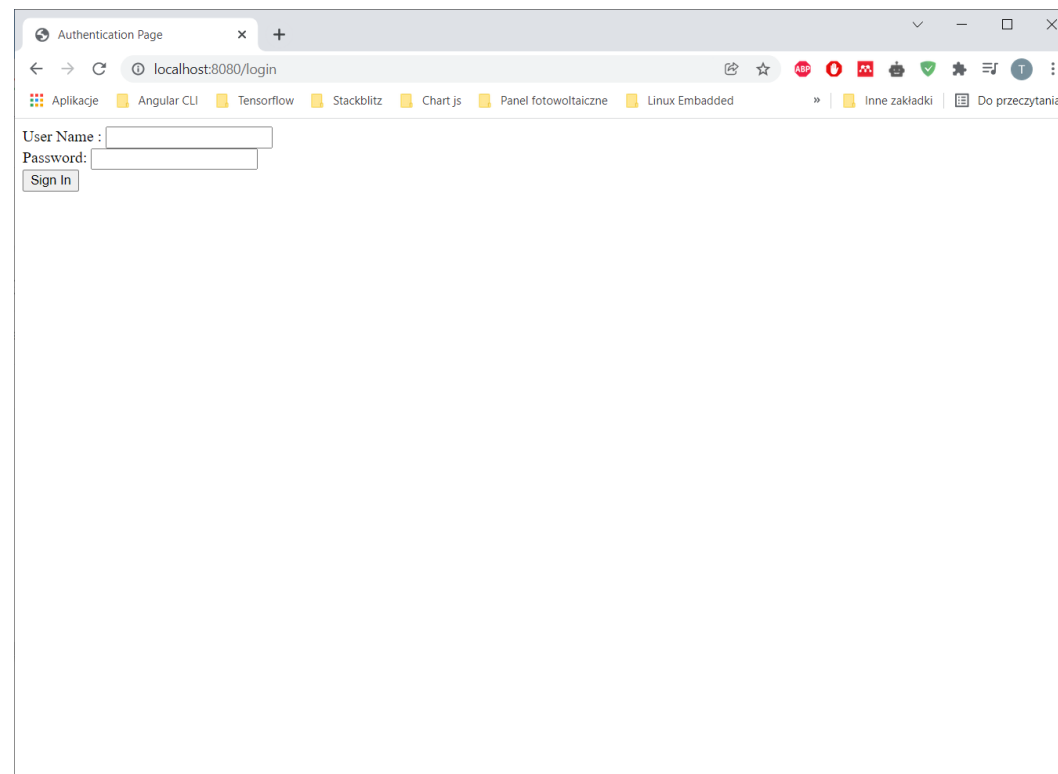
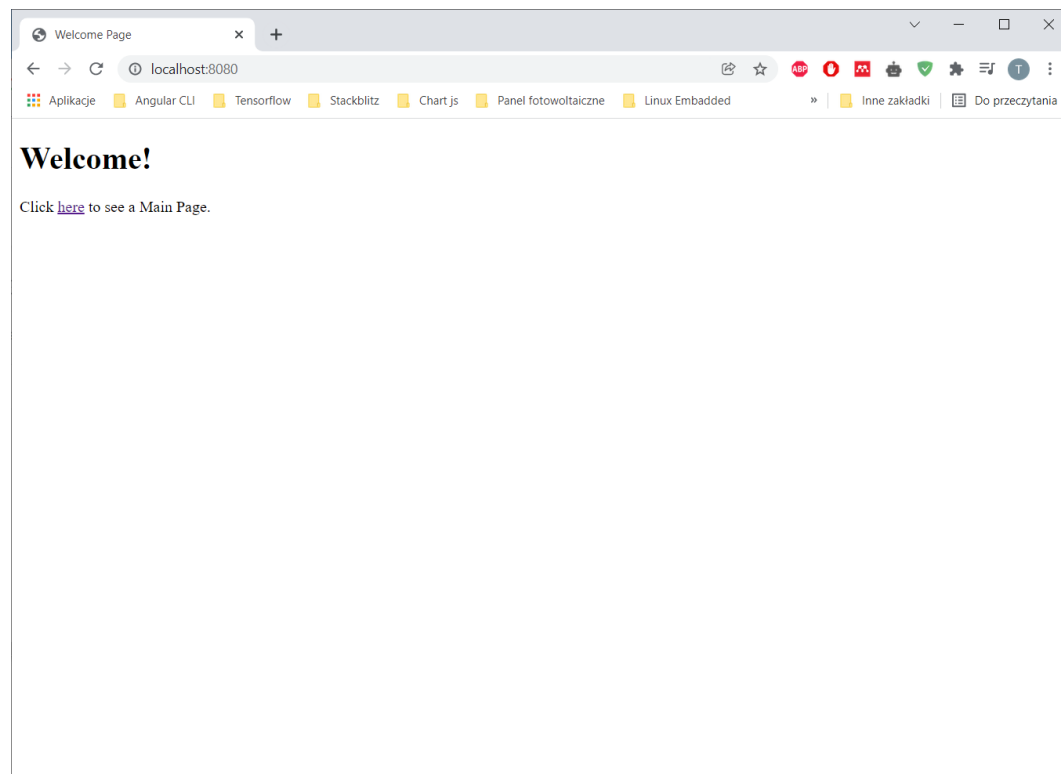
Tworzenie prostej aplikacji webowej ze stroną logowania

Kod dla pliku **login.html** (do skopiowania):

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-springsecurity3">
<head>
  <title>Authentication Page</title>
</head>
<body>
<div th:if="{param.error}">
  Invalid username and password.
</div>
<div th:if="{param.logout}">
  You have been logged out.
</div>
<form th:action="@{/login}" method="post">
  <div><label> User Name : <input type="text" name="username"/> </label></div>
  <div><label> Password: <input type="password" name="password"/> </label></div>
  <div><input type="submit" value="Sign In"/></div>
</form>
</body>
</html>
```

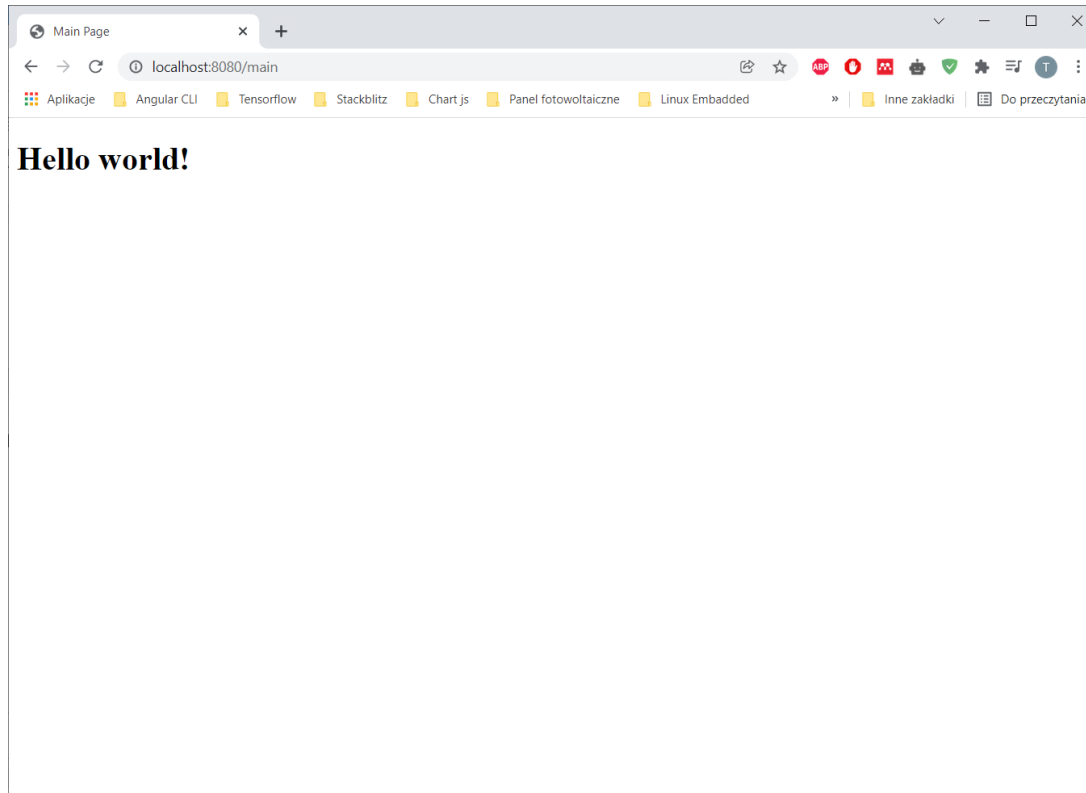
Tworzenie prostej aplikacji webowej ze stroną logowania

Po uruchomieniu aplikacja wygląda następująco:



Tworzenie prostej aplikacji webowej ze stroną logowania

Po uruchomieniu aplikacja wygląda następująco:



Tworzenie prostej aplikacji webowej ze stroną logowania

Szablon Thymeleaf został wykorzystany do zarządzania częścią logowania, tzn. przechwytywania loginu i hasła. Spring Security z kolei zapewnia filtr, który przechwytuje żądanie i uwierzytelnia użytkownika. Jeśli uwierzytelnienie nie powiedzie się, to zostanie wyświetlony odpowiedni komunikat np. złe hasło itd. Po poprawnym zalogowaniu nastąpi przekierowanie na stronę główną.

Do strony głównej **/main (main.html)** należy jeszcze dodać opcję umożliwiającą wylogowanie.

Tworzenie prostej aplikacji webowej ze stroną logowania

Uaktualniony kod dla pliku **main.html**:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-
springsecurity3">
<head>
  <title>Main Page</title>
</head>
<body>
<h1 th:inline="text">Hello [[${#httpServletRequest.remoteUser}]]!</h1>
<form th:action="@{/logout}" method="post">
  <input type="submit" value="Sign Out"/>
</form>
</body>
</html>
```

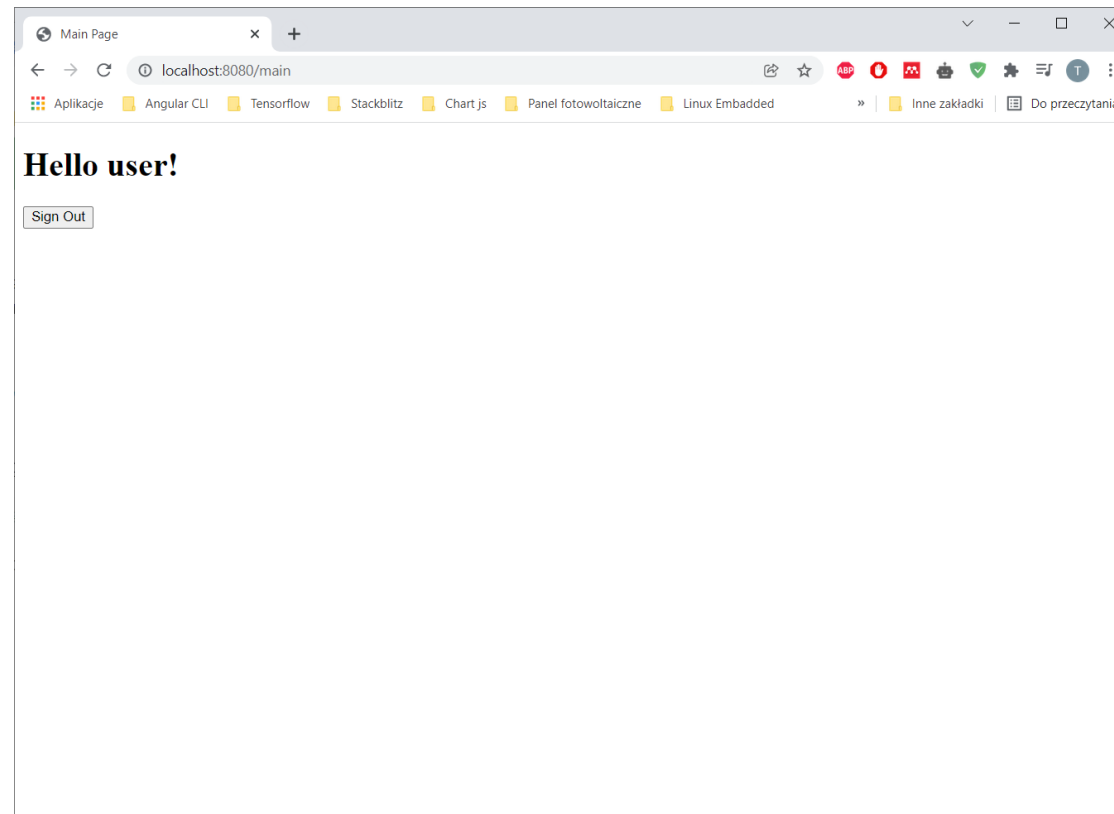
Tworzenie prostej aplikacji webowej ze stroną logowania

Uaktualniony kod dla pliku **main.html** (do skopiowania):

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-springsecurity3">
<head>
  <title>Main Page</title>
</head>
<body>
<h1 th:inline="text">Hello [[${#httpServletRequest.remoteUser}]]!</h1>
<form th:action="@{/logout}" method="post">
  <input type="submit" value="Sign Out"/>
</form>
</body>
</html>
```

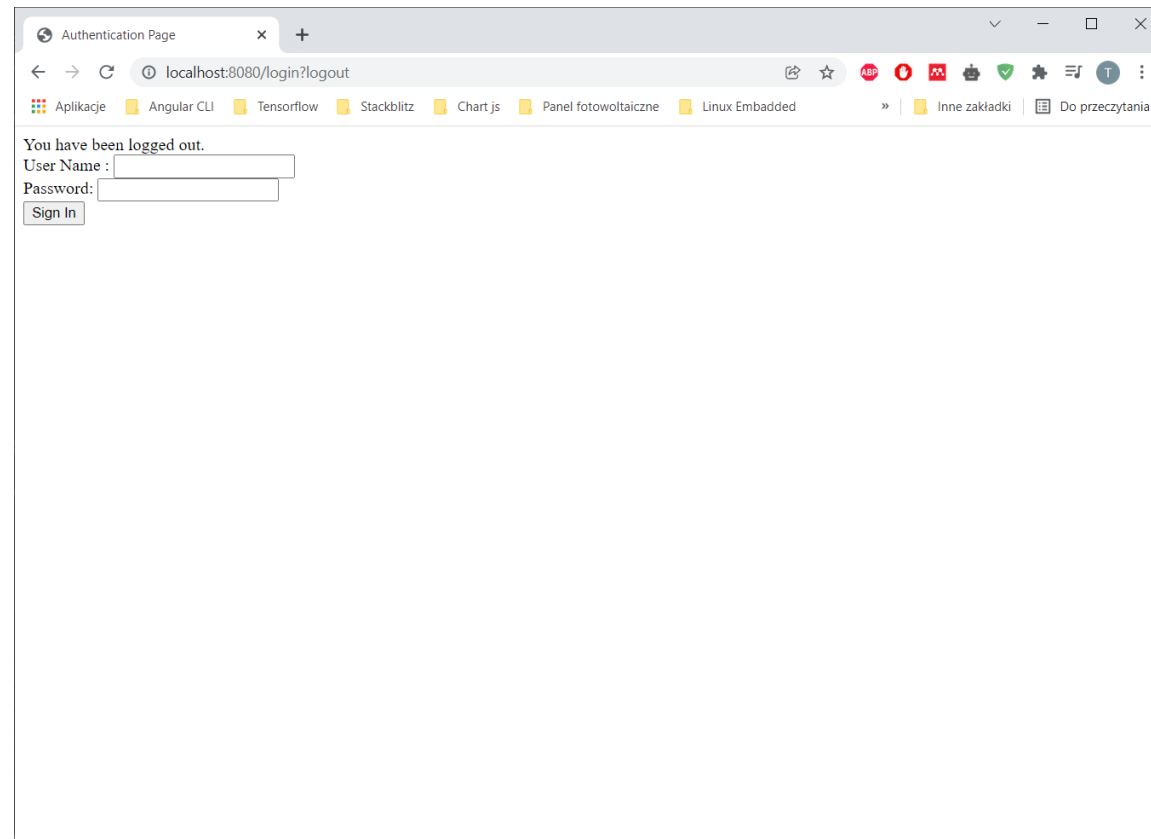
Tworzenie prostej aplikacji webowej ze stroną logowania

Po aktualizacji kodu, strona **main.html** wygląda następująco:



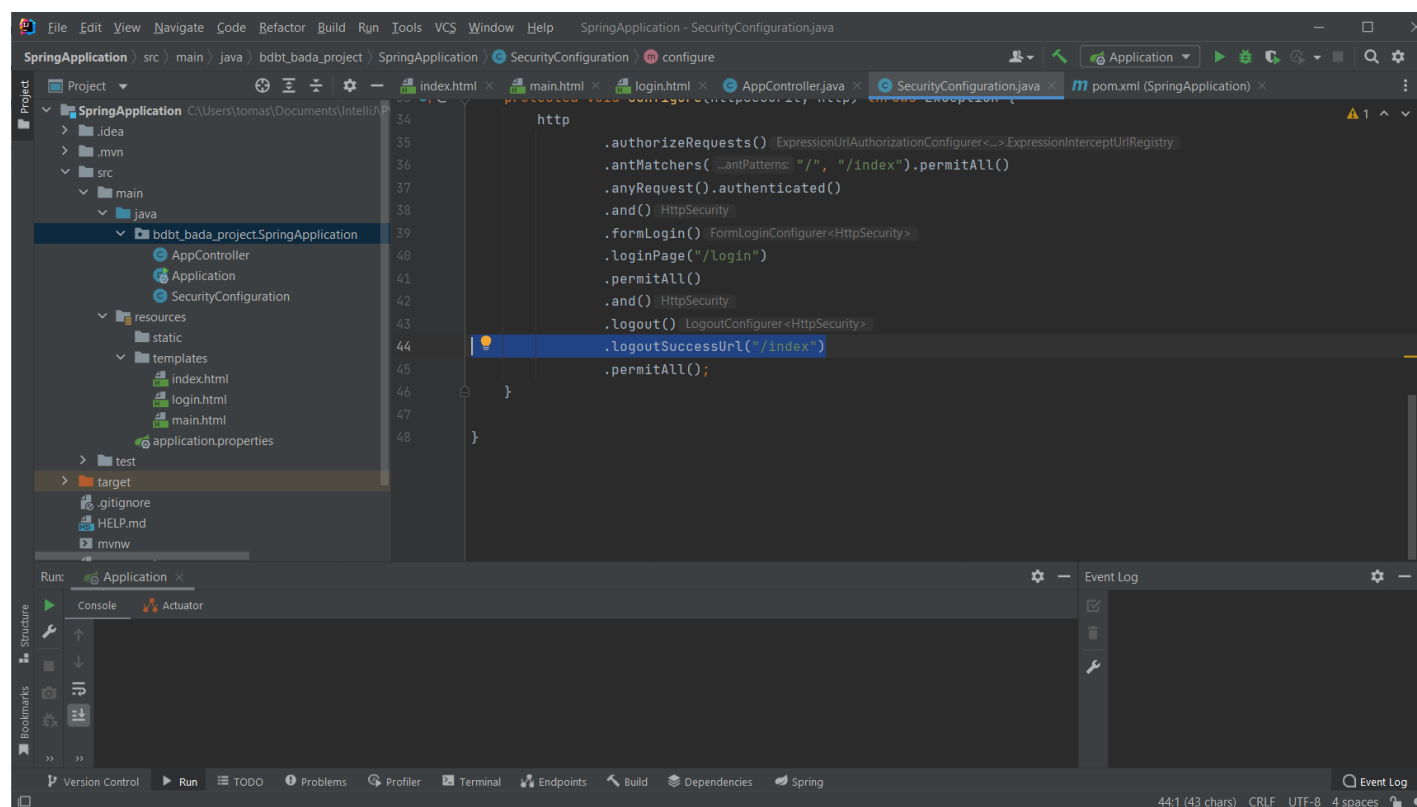
Tworzenie prostej aplikacji webowej ze stroną logowania

Po wylogowaniu nastąpi przekierowanie do strony logowania z komunikatem „You have been logged out.”



Tworzenie prostej aplikacji webowej ze stroną logowania

Aby umożliwić przekierowanie np. do strony powitalnej **index.html** należy dokonać małej modyfikacji w pliku **SecurityConfiguration.java**.



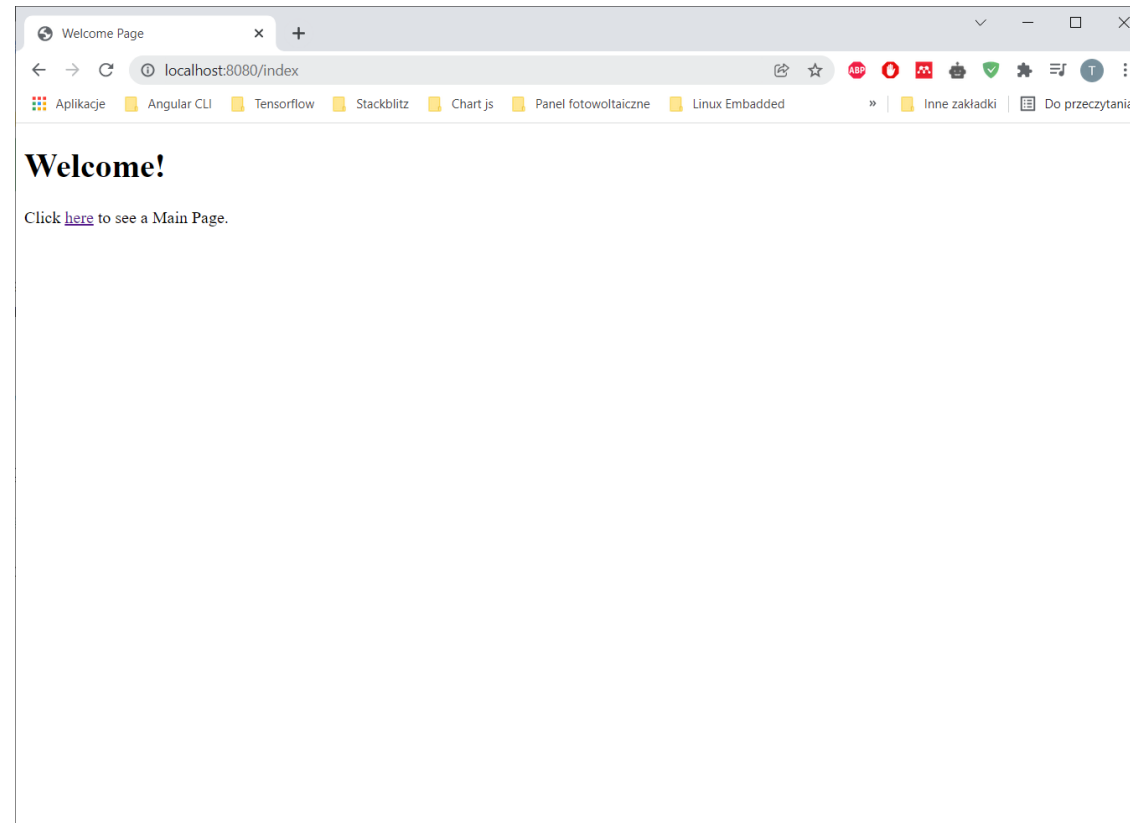
Tworzenie prostej aplikacji webowej ze stroną logowania

Modyfikacja dla pliku **SecurityConfiguration.java**. Aby zmienić przekierowanie na inną stronę (w tym przypadku **/index**) należy dodać czerwony fragment kodu (do skopiowania):

```
.logout()  
.logoutSuccessUrl("/index")  
.permitAll();
```


Tworzenie prostej aplikacji webowej ze stroną logowania

Po tej modyfikacji nastąpi przekierowanie do strony powitalnej:

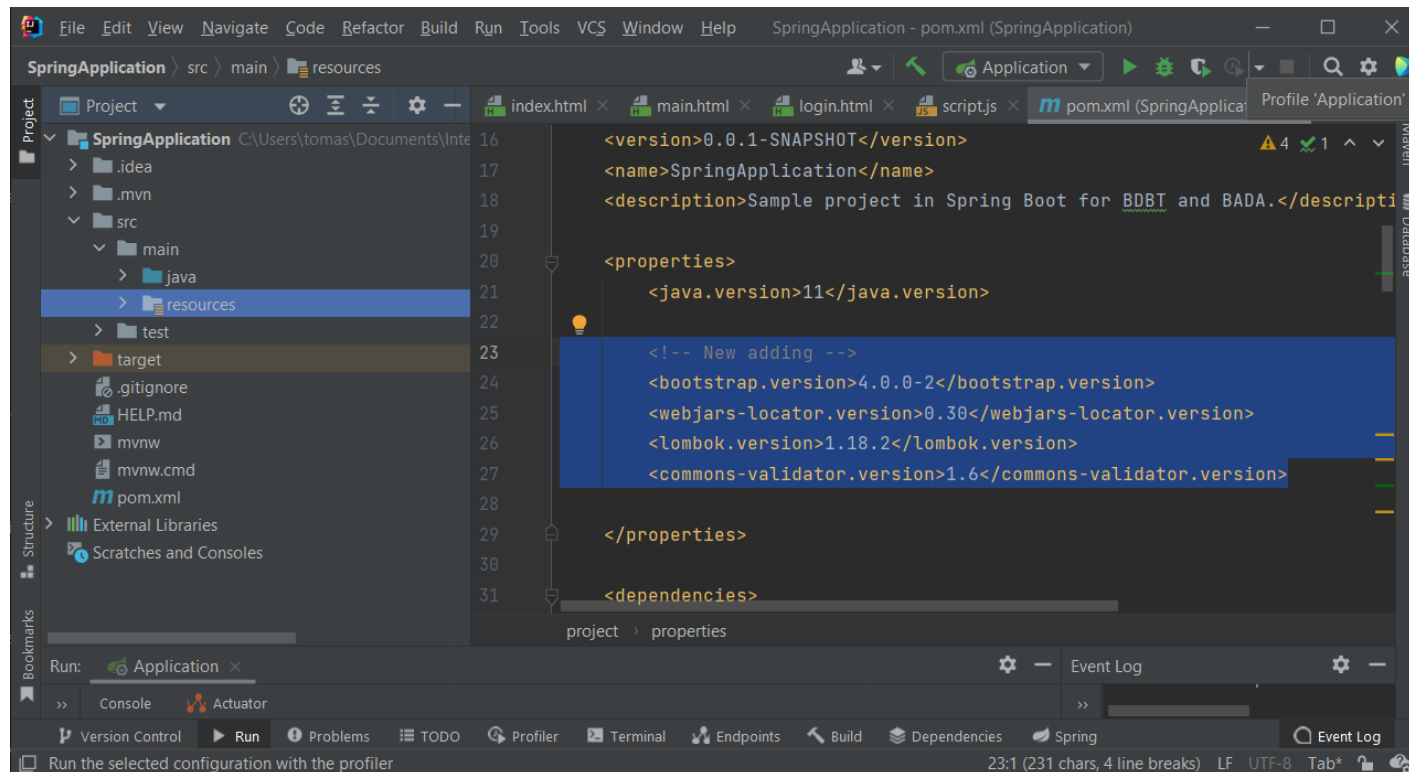


3

MODYFIKACJA APLIKACJI – BOOTSTRAP (CSS)

Modyfikacja aplikacji – Bootstrap (CSS)

Pierwszym etapem modyfikacji stworzonej aplikacji jest aktualizacja pliku „pom.xml”. Należy dodać w zaznaczonym miejscu poniższy fragment kodu.



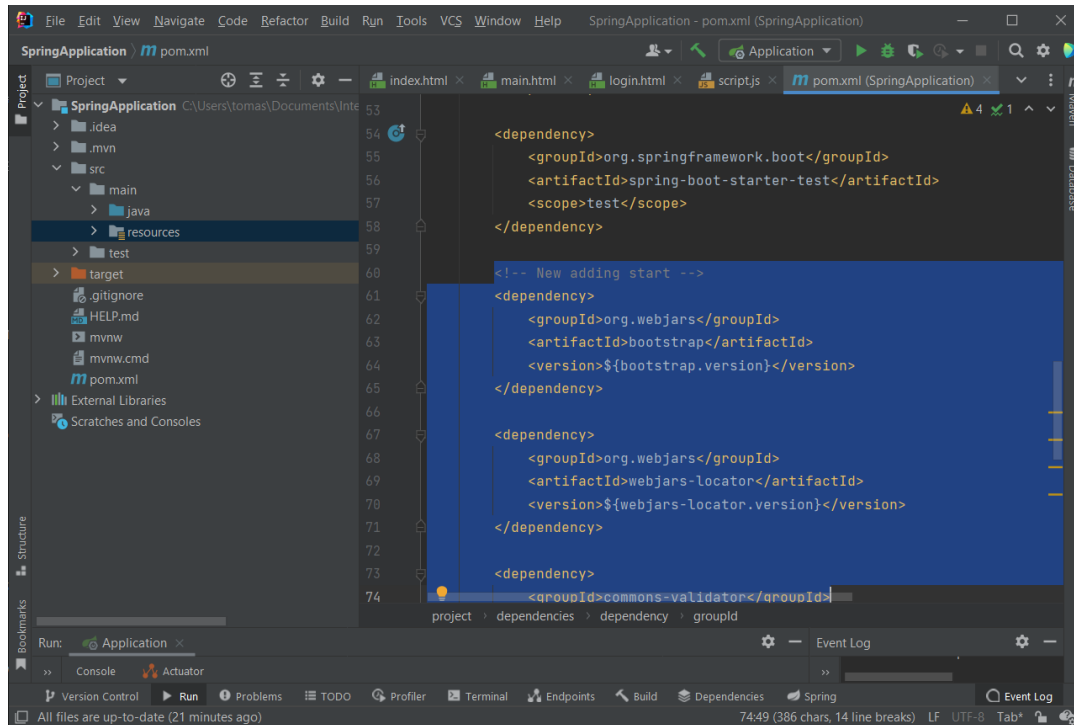
Modyfikacja aplikacji – Bootstrap (CSS)

Fragment kodu dla pliku **pom.xml** (do skopiowania):

```
<!-- New adding -->  
<bootstrap.version>4.0.0-2</bootstrap.version>  
<webjars-locator.version>0.30</webjars-locator.version>  
<lombok.version>1.18.2</lombok.version>  
<commons-validator.version>1.6</commons-validator.version>
```

Modyfikacja aplikacji – Bootstrap (CSS)

W tym samym pliku „pom.xml” w dependencies należy dodać nowe zależności zgodnie z poniższym rysunkiem:



The screenshot shows the IntelliJ IDEA IDE with the 'pom.xml' file open. The file is part of a project named 'SpringApplication'. The 'dependencies' section is highlighted in blue, showing the following XML code:

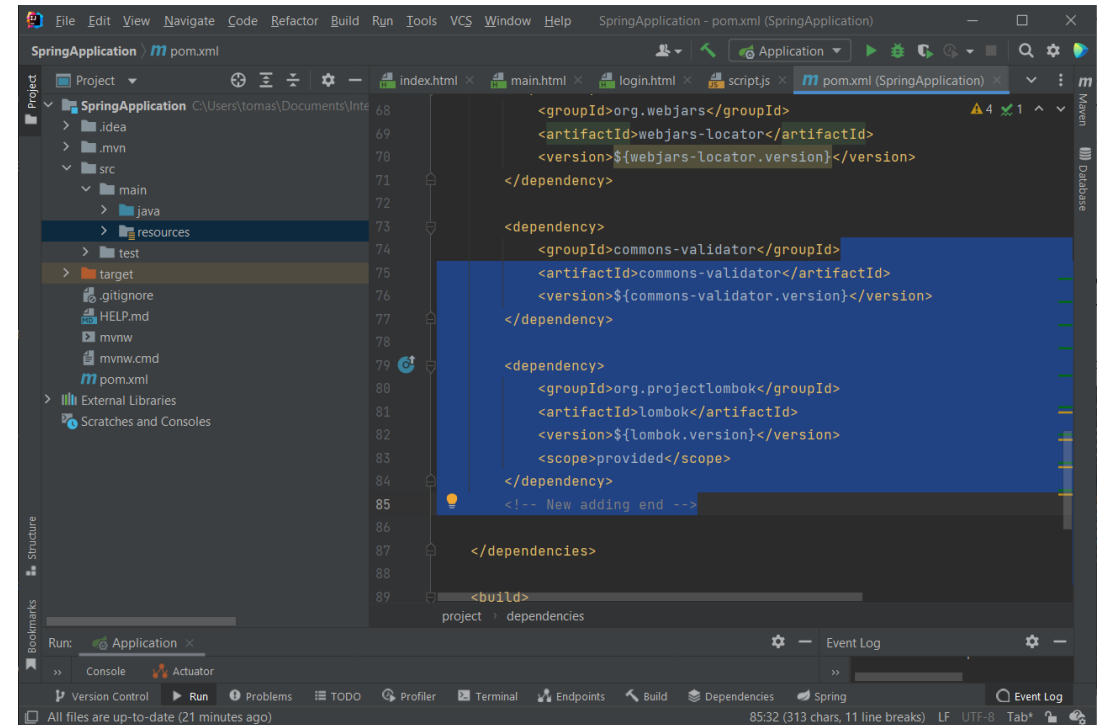
```
<!-- New adding start -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>bootstrap</artifactId>
  <version>${bootstrap.version}</version>
</dependency>

<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>webjars-locator</artifactId>
  <version>${webjars-locator.version}</version>
</dependency>

<dependency>
  <groupId>commons-validator</groupId>
```

The status bar at the bottom indicates the file is 74:49 (386 chars, 14 line breaks) in UTF-8 encoding.



The screenshot shows the IntelliJ IDEA IDE with the 'pom.xml' file open. The file is part of a project named 'SpringApplication'. The 'dependencies' section is highlighted in blue, showing the following XML code:

```
<groupId>org.webjars</groupId>
<artifactId>webjars-locator</artifactId>
<version>${webjars-locator.version}</version>
</dependency>

<dependency>
  <groupId>commons-validator</groupId>
  <artifactId>commons-validator</artifactId>
  <version>${commons-validator.version}</version>
</dependency>

<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>${lombok.version}</version>
  <scope>provided</scope>
</dependency>
<!-- New adding end -->
</dependencies>
```

The status bar at the bottom indicates the file is 85:32 (313 chars, 11 line breaks) in UTF-8 encoding.

Modyfikacja aplikacji – Bootstrap (CSS)

Fragment kodu dla pliku **pom.xml** (zależności do skopiowania):

```
<!-- New adding start -->
<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>bootstrap</artifactId>
  <version>${bootstrap.version}</version>
</dependency>

<dependency>
  <groupId>org.webjars</groupId>
  <artifactId>webjars-locator</artifactId>
  <version>${webjars-locator.version}</version>
</dependency>
```

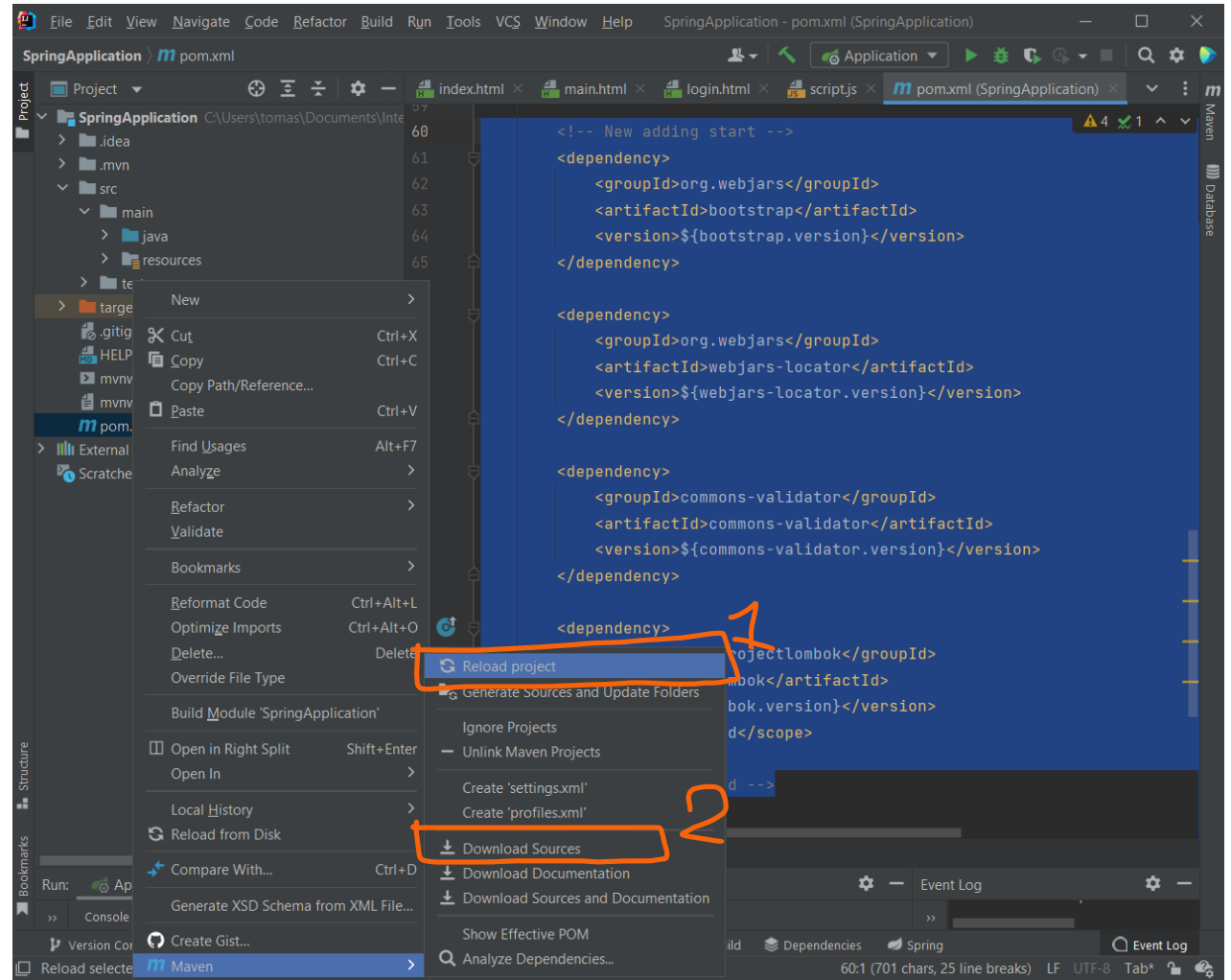
```
<dependency>
  <groupId>commons-validator</groupId>
  <artifactId>commons-validator</artifactId>
  <version>${commons-validator.version}</version>
</dependency>

<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>${lombok.version}</version>
  <scope>provided</scope>
</dependency>
<!-- New adding end -->
```

Modyfikacja aplikacji – Bootstrap (CSS)

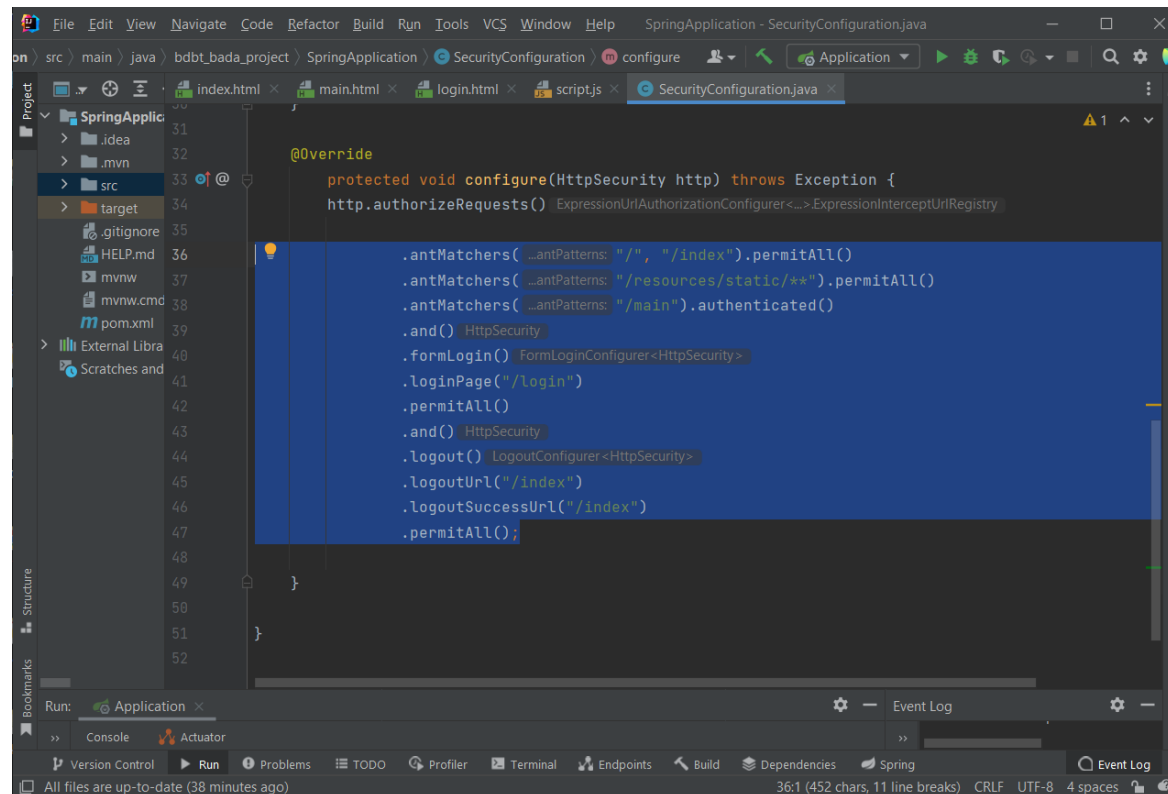
Po aktualizacji pliku **pom.xml** należy wybrać prawy klawisz myszy i z rozwijanej listy Maven wybrać „Reload project” (1) a następnie „Download Sources” (2).

W ten sposób nastąpi instalacja i pobranie wszystkich nowych zależności.



Modyfikacja aplikacji – Bootstrap (CSS)

Kolejnym etapem jest modyfikacja w pliku **SecurityConfiguration.java**. Należy uaktualnić (zmodyfikować) kod w zaznaczonym miejscu (na rysunku znajduje się już kod z niezbędną modyfikacją):



Modyfikacja aplikacji – Bootstrap (CSS)

Fragment kodu dla pliku **SecurityConfiguration.java** (zależności do skopiowania/podmiany):

```
.antMatchers("/", "/index").permitAll()  
.antMatchers("/resources/static/**").permitAll()  
.antMatchers("/main").authenticated()  
.and()  
.formLogin()  
.loginPage("/login")  
.permitAll()  
.and()  
.logout()  
.logoutUrl("/index")  
.logoutSuccessUrl("/index")  
.permitAll();
```

Modyfikacja aplikacji – Bootstrap (CSS)

Kolejnym etapem jest modyfikacja plików: **index.html**, **main.html** i **login.html**.

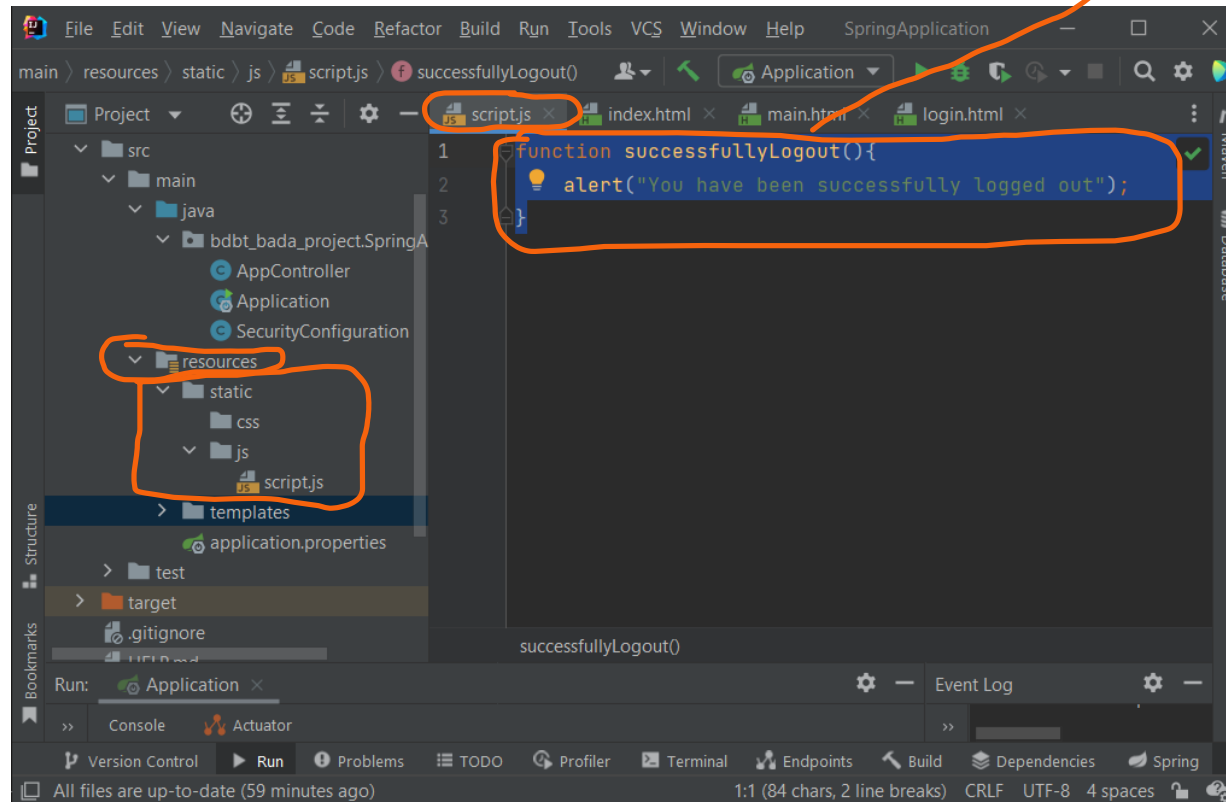
Ze względu na znacznie większą objętość kodu w każdym z plików, zrezygnowano z zamieszczania kodów do każdego pliku na slajdach w tej prezentacji. Zawartości kodów do skopiowania znajdują się w dodatkowo załączonych plikach (pliki te zostały spakowane w pliku pt. ZałącznikiPlikiHtml.7z):

- index.html,
- main.html,
- login.html.

UWAGA! Kody zamieszczone w powyższych plikach należy zamienić z tymi z pierwszej części. Innymi słowy treści ze starych plików można w całości usunąć, natomiast w ich miejsce należy wkleić nową treść.

Modyfikacja aplikacji – Bootstrap (CSS)

Ostatnim etapem modyfikowanej aplikacji jest dodanie folderu **static** (w resources), a w nim folderów **css** i **js** (a w nim stworzenie pliku **script.js**). W pliku **script.js** należy dodać zaznaczony kod zgodnie z poniższym obrazkiem.



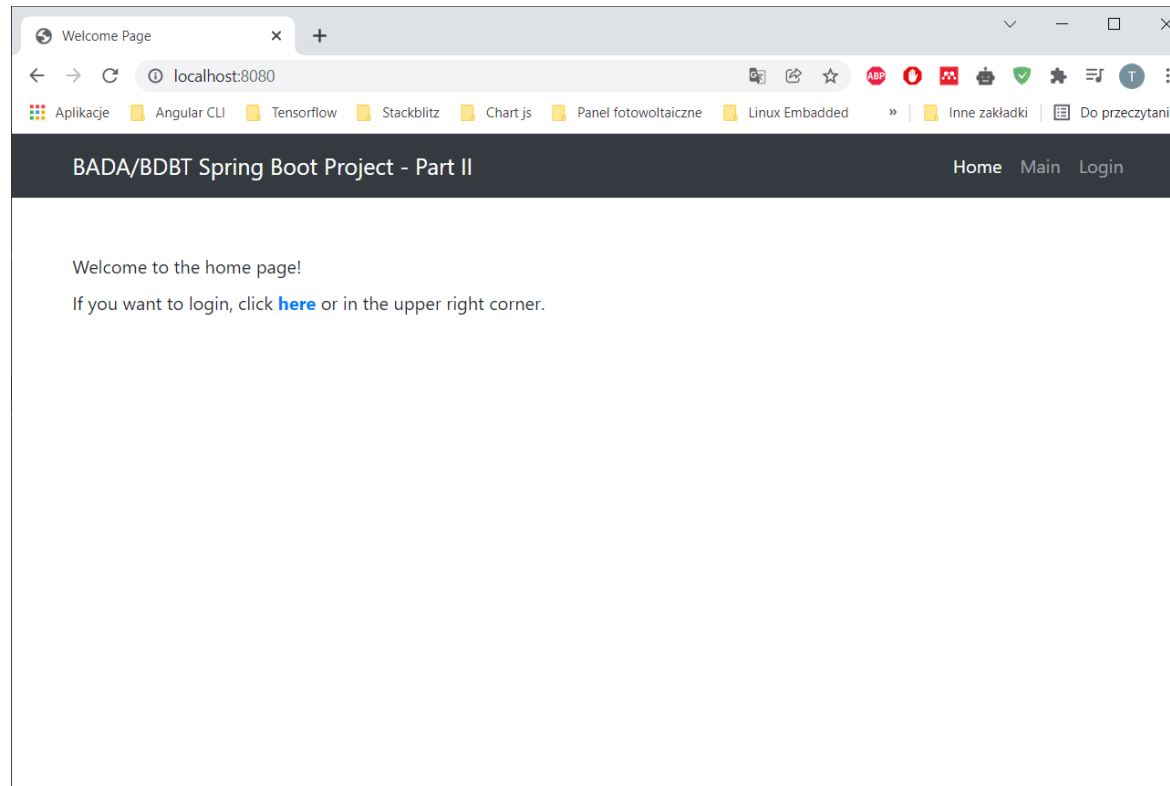
Modyfikacja aplikacji – Bootstrap (CSS)

Kod dla pliku **script.js** (do skopiowania):

```
function successfullyLogout(){  
    alert("You have been successfully logged out");  
}
```

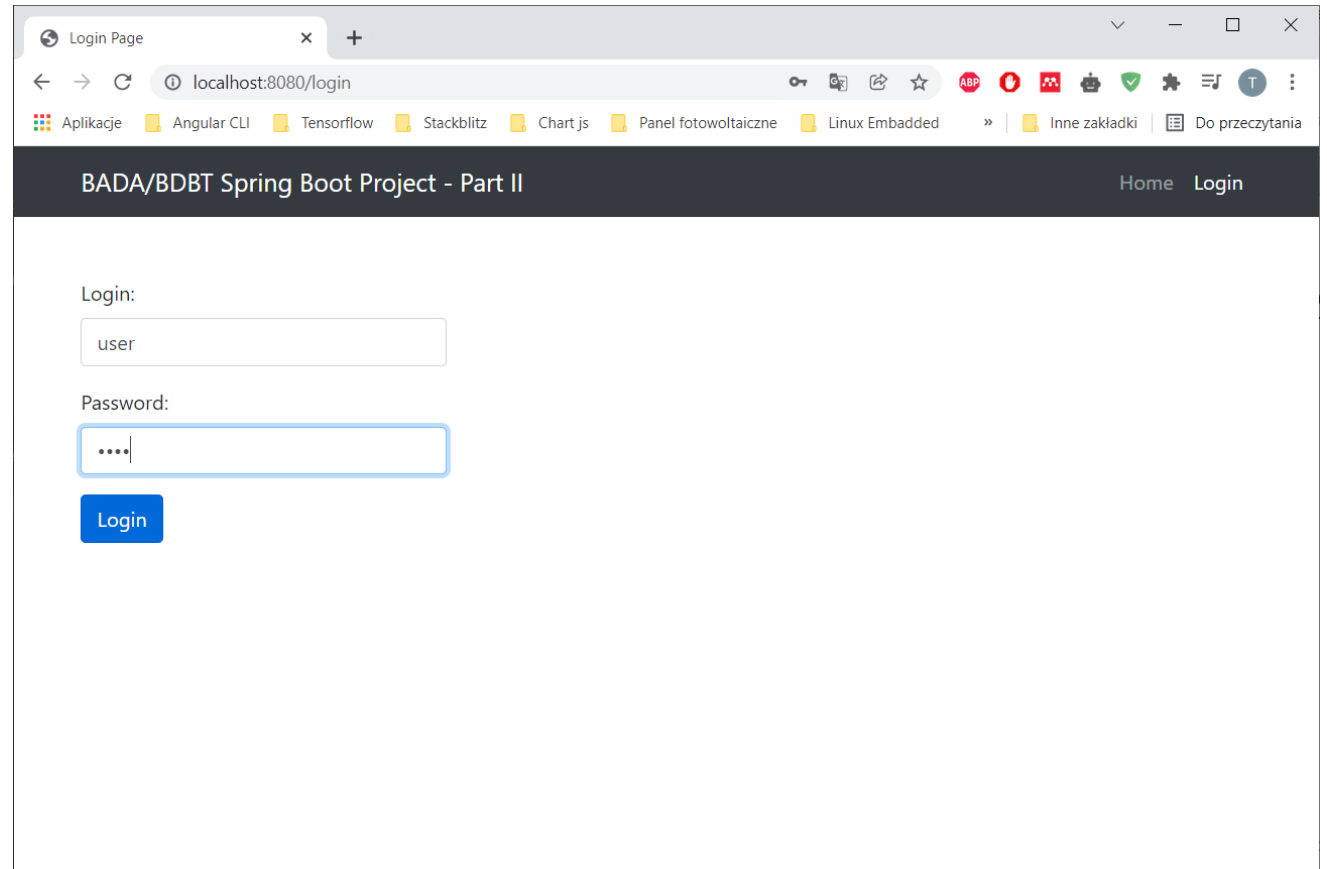
Modyfikacja aplikacji – Bootstrap (CSS)

Po takich modyfikacjach aplikacja będzie się prezentowała następująco (**widok na stronę powitalną**, wybór napisu „here” lub kliknięcie w napis „Login” spowoduje przekierowanie do strony logowania:



Modyfikacja aplikacji – Bootstrap (CSS)

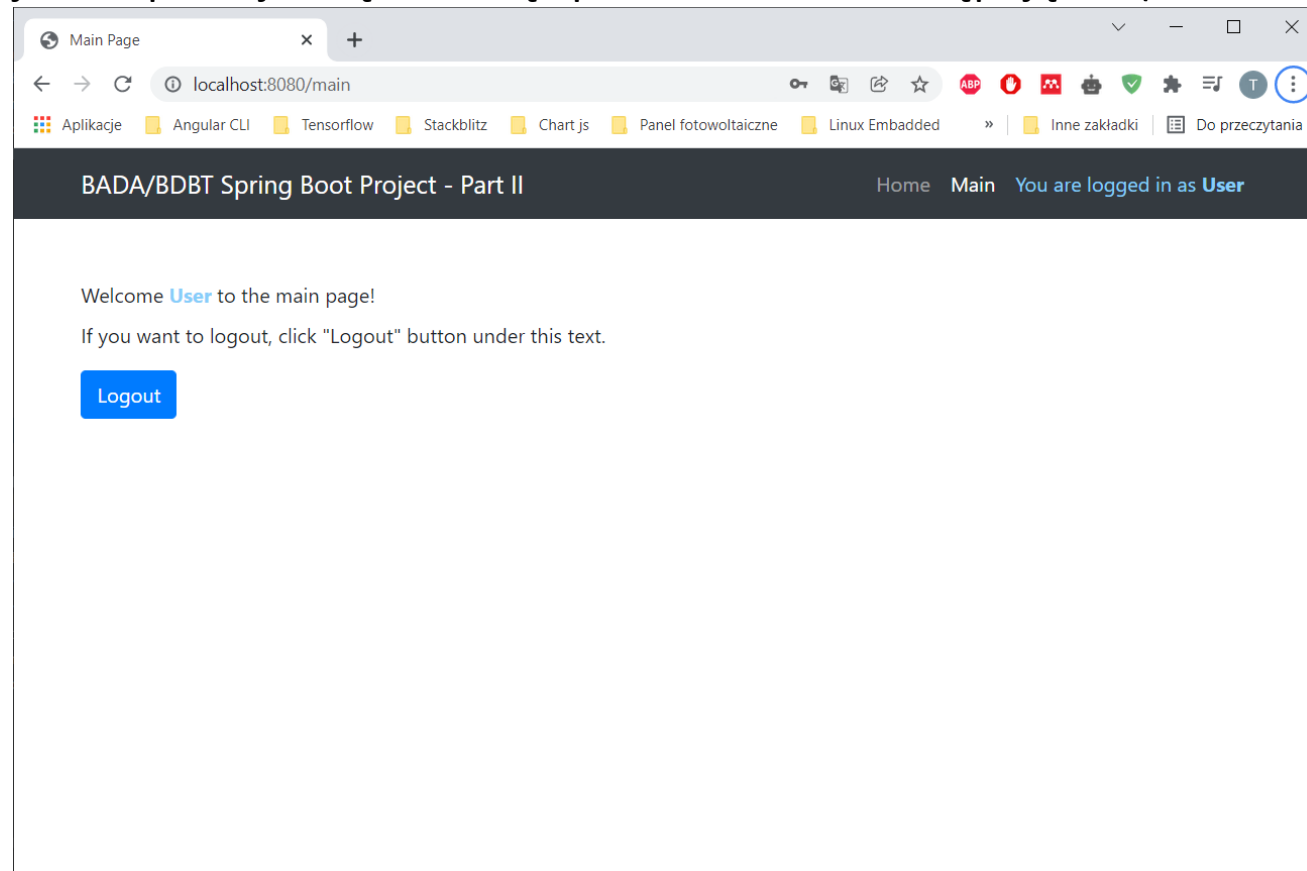
Po takich modyfikacjach aplikacja będzie się prezentowała następująco (**strona logowania**, po kliknięciu w button nastąpi zalogowanie na stronę główną):



The screenshot shows a web browser window with the title 'Login Page' and the address bar displaying 'localhost:8080/login'. The browser's bookmark bar includes 'Aplikacje', 'Angular CLI', 'Tensorflow', 'Stackblitz', 'Chart.js', 'Panel fotowoltaiczne', 'Linux Embadded', and 'Inne zakładki'. The page content features a dark header with the text 'BADA/BDBT Spring Boot Project - Part II' and links for 'Home' and 'Login'. Below the header, the login form consists of a 'Login:' label, a text input field containing 'user', a 'Password:' label, a password input field with masked characters '....', and a blue 'Login' button.

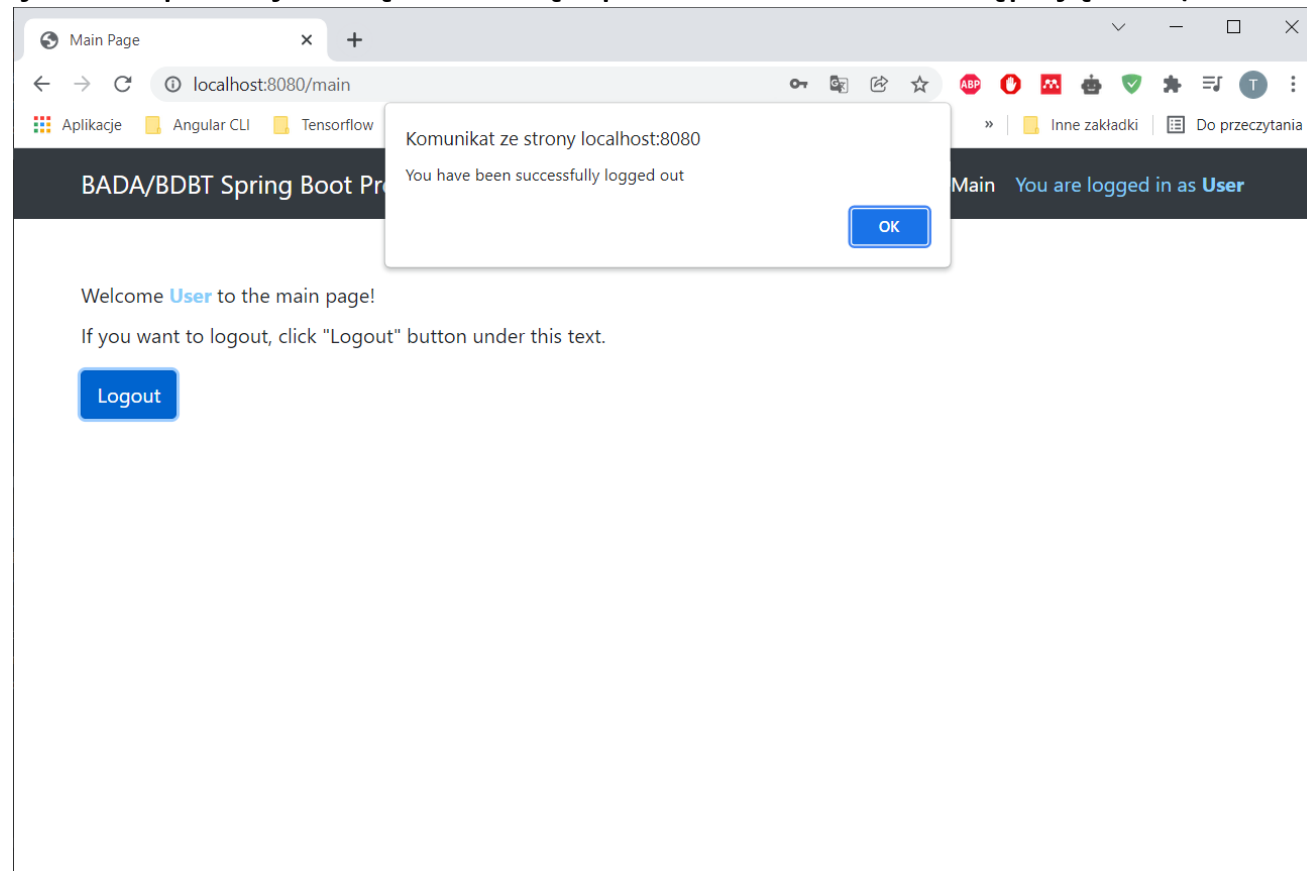
Modyfikacja aplikacji – Bootstrap (CSS)

Po takich modyfikacjach aplikacja będzie się prezentowała następująco (**strona główna** dostępna po zalogowaniu):



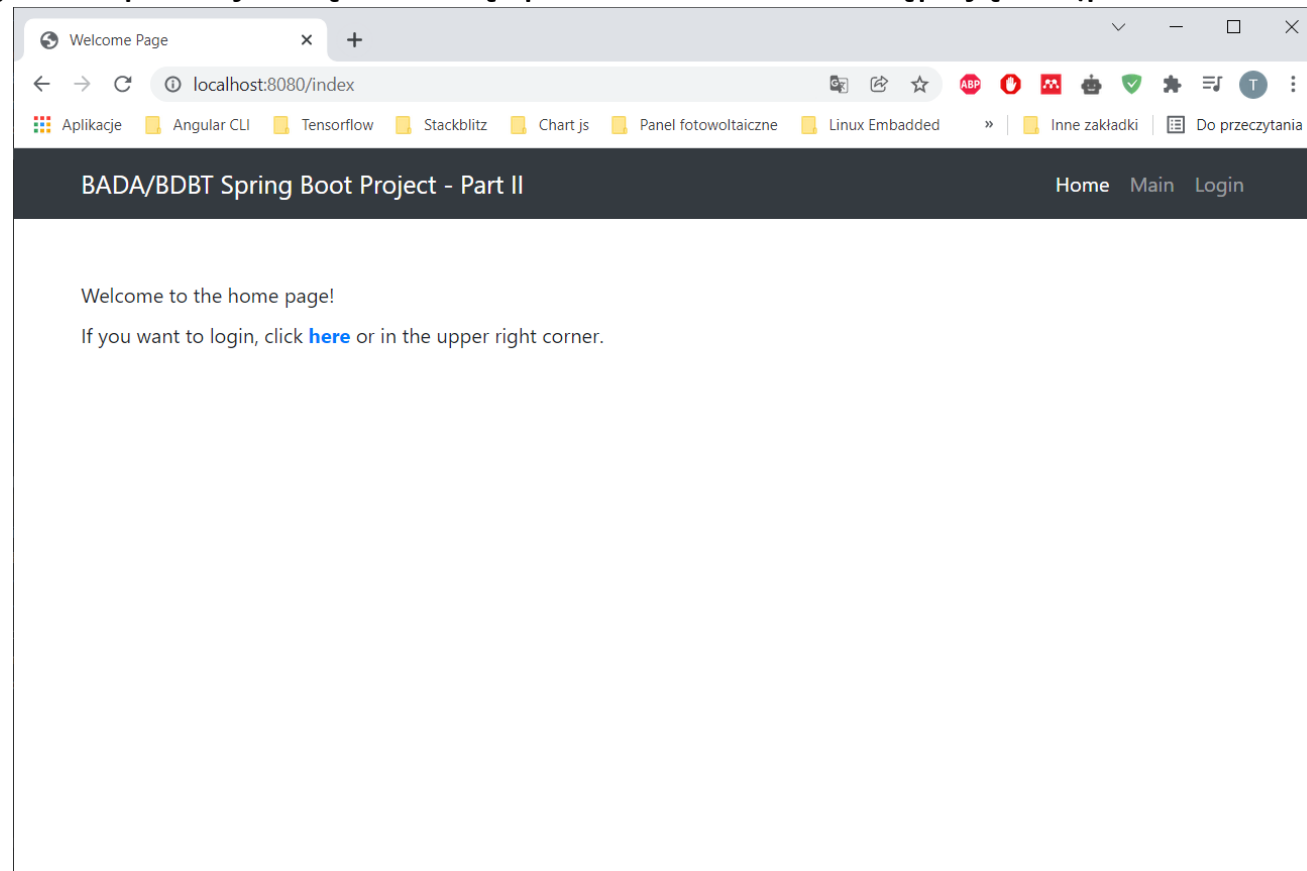
Modyfikacja aplikacji – Bootstrap (CSS)

Po takich modyfikacjach aplikacja będzie się prezentowała następująco (komunikat po poprawnym wylogowaniu):



Modyfikacja aplikacji – Bootstrap (CSS)

Po takich modyfikacjach aplikacja będzie się prezentowała następująco (powrót do strony powitalnej po wylogowaniu):



Modyfikacja aplikacji – Bootstrap (CSS)

Zmodyfikowaną aplikację oraz kod należy dokładnie przeanalizować w celach naukowych.

**Politechnika
Warszawska**

Koniec

